Proyecto RISK: documento de diseño incremental de los componentes 1 y 2

Saúl Leonardo Cruz Romero[1*], Isabella Hermosa Losada[1] and Alejandro Macías Barrios[1]

[1*] Estructuras de Datos, Pontificia Universidad Javeriana, Bogotá D.C., Colombia .

*Corresponding author(s). E-mail(s): sa.cruz@javeriana.edu.co;
Contributing authors: ishermosa@javeriana.edu.co;
amaciasb@javeriana.edu.co;

Resumen

Este documento presenta el diseño incremental del proyecto RISK hasta la Entrega 2. Se conserva y actualiza el componente de gestión de partidas, inicialmente implementado con estructuras lineales, y se incorpora el almacenamiento en texto y la compresión y recuperación de archivos binarios mediante codificación de Huffman y estructuras jerárquicas.
Se describen los comandos del sistema, las especificaciones de los Tipos Abstractos de Datos (TAD), sus relaciones y las decisiones de implementación. También se presentan los planes y resultados de pruebas de los comandos atacar y guardar_comprimido, distinguiendo los resultados registrados de las evidencias de ejecución verificadas.

Keywords: RISK, estructuras de datos, TAD, árboles binarios, montículos, Huffman, validación, pruebas

1. Acta de evaluación de la Entrega 1

En la sustentación de la Entrega 1, el equipo obtuvo una calificación de **5,0**. Según el registro compartido por el equipo, no se comunicaron correcciones específicas por escrito. Por tanto, no se atribuyen al docente comentarios textuales que no estén disponibles.

Las modificaciones posteriores corresponden principalmente a la ampliación del proyecto para el Componente 2 y a la revisión de compatibilidad de la carga de partidas guardadas.

En particular, un archivo producido durante una partida puede contener cantidades de unidades distintas de la distribución de apertura y debe poder recuperarse sin aplicar nuevamente el total inicial como condición de la partida en curso.

2. Descripción del problema

RISK es un juego de mesa por turnos en el que participan entre tres y seis jugadores. Cada jugador se identifica mediante un nombre y un color diferentes.

La distribución de apertura incluye 35 unidades para tres participantes, 30 para cuatro, 25 para cinco y 20 para seis. Durante la partida, estas cantidades cambian mediante conquistas, pérdidas de unidades y refuerzos [1].

El tablero está formado por 42 territorios agrupados en seis continentes: América del Norte, América del Sur, Europa, África, Asia y Oceanía.

Cada territorio almacena su propietario, cantidad de unidades y conexiones con territorios vecinos. Estas relaciones permiten comprobar los ataques y las fortificaciones válidas.

La Figura 1 presenta el mapa del juego, incluido originalmente en la documentación de la Entrega 1.

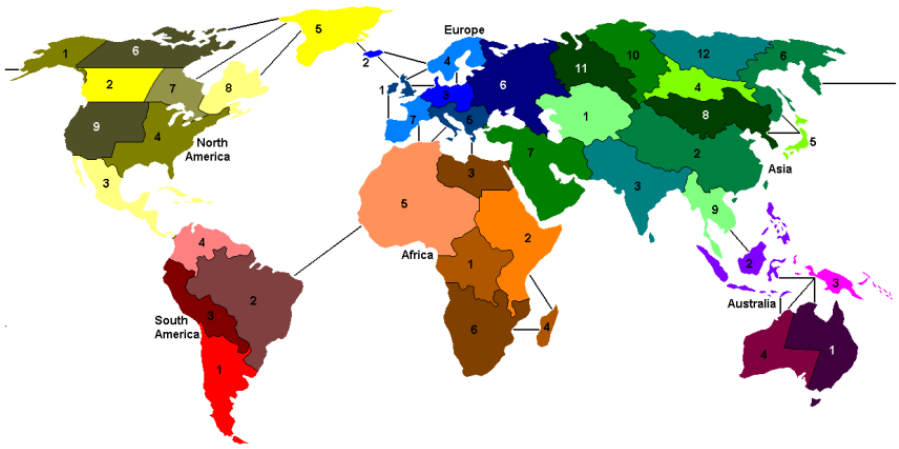


Figura 1 Tablero de RISK. Tomado del enunciado oficial del proyecto, que acredita la imagen a Demonaire bajo licencia CC BY-SA 3.0 [1].

El turno comprende tres etapas: obtención y ubicación de unidades, ataque y fortificación.

Durante un ataque, los jugadores lanzan los dados permitidos según las unidades disponibles. Los resultados se ordenan de mayor a menor y se comparan por posición. Cuando dos dados tienen el mismo valor, el empate favorece al defensor.

Las cartas permiten realizar intercambios para obtener unidades adicionales. La condición de victoria obligatoria consiste en controlar los 42 territorios del tablero.

La variante de misiones secretas se considera una ampliación opcional y no forma parte de los componentes obligatorios implementados hasta la Entrega 2.

3. Alcance y organización del sistema

El Componente 1 administra los jugadores, el tablero, las cartas, los ataques y el desarrollo de los turnos.

El Componente 2 incorpora el almacenamiento del estado de una partida en archivos de texto y en archivos binarios comprimidos mediante el algoritmo de Huffman.

Ambos componentes funcionan mediante la misma interfaz de comandos en consola. Las operaciones nuevas reutilizan las estructuras existentes del juego.

El Componente 3, relacionado con estructuras de grafos, corresponde a la siguiente etapa del proyecto y no se presenta como implementado en este documento [1].

El diseño se organiza mediante los TAD utilizados, sus relaciones, los procedimientos de los comandos y los planes de pruebas.

Las especificaciones de TAD siguen el formato presentado en las diapositivas de clase: propósito, estado, condiciones, interfaz, precondiciones y poscondiciones.

El diseño original del equipo se conservó como punto de partida. La descripción de las funciones auxiliares y del procedimiento principal conserva el criterio trabajado en el Taller 2 de estructuras lineales [3, 5].

4. Componente 1: gestión de la partida

El Componente 1 administra la información necesaria para inicializar y desarrollar una partida de RISK. Los datos se organizan mediante estructuras lineales y TAD del material de clase [4] que representan jugadores, territorios, continentes, cartas, dados, ataques, turnos y el estado general de la partida.

El sistema controla el orden de las tres etapas del turno. Primero se obtienen y distribuyen las unidades, después se realizan los ataques y, finalmente, se permite fortificar. Al terminar la última etapa, el turno pasa al siguiente jugador que todavía conserva territorios.

Las operaciones se ejecutan desde una interfaz de comandos en consola. A continuación se describen sus entradas, condiciones, procesos y resultados, siguiendo la organización del documento original de la Entrega 1.

4.1. Descripción de los comandos

INICIALIZAR

Forma de uso: `inicializar nombre_archivo`

Permite cargar una partida a partir de un archivo que contiene los jugadores y la distribución de propietarios y unidades de los 42 territorios.

Entrada. Recibe el nombre o la ruta de un archivo de texto que contiene la configuración de la partida.

Por ejemplo, `inicializar partida.txt`.

Condiciones. No debe existir una partida previamente inicializada. El archivo debe poder abrirse y contener entre tres y seis jugadores, con nombres de máximo ocho caracteres, colores válidos y sin repeticiones.

Deben aparecer los 42 territorios del tablero, cada uno con un código válido, un propietario registrado y una cantidad válida de unidades.

En el enunciado, la distribución de apertura corresponde a 35, 30, 25 o 20 unidades por jugador, según participen tres, cuatro, cinco o seis jugadores. La ampliación para recuperar partidas guardadas después de varios turnos se documenta en el Componente 2.

Proceso. Primero se comprueba que el archivo de texto exista y pueda leerse.

El contenido se procesa como una entrada de texto. Se validan los jugadores y territorios antes de aceptar la partida. Cuando la información es correcta, se configura el tablero, se registran los participantes, se inicia el turno y se prepara el mazo.

El primer jugador escrito en el archivo ocupa la posición inicial del turno.

Resultados. La operación puede retornar `COMANDO_CORRECTO`, `ARCHIVO_VACIO`, `ARCHIVO_ERRONEO`, `ARCHIVO_SIN_FORMATO` o `JUEGO_EN_CURSO`.

Si se produce un error de lectura o validación, no se acepta una partida incompleta.

OBTENER_UNIDADES

Forma de uso: `obtener_unidades nombre_jugador`

Calcula las unidades que recibe un jugador durante su turno y permite distribuirlas entre territorios propios.

Entrada. Recibe el nombre de un jugador. Durante la ejecución, el programa solicita información sobre intercambios de cartas y pares de datos formados por código de territorio y cantidad de unidades.

Condiciones. La partida debe estar activa. El jugador debe existir, tener el turno y encontrarse en la etapa de obtención de unidades.

Las cantidades ingresadas deben ser enteras positivas, no superar las unidades disponibles y asignarse a territorios pertenecientes al jugador.

Cálculo. Las unidades obtenidas se calculan teniendo en cuenta la división entera de los territorios controlados entre tres, las bonificaciones por continentes completos y los intercambios válidos de cartas.

Los intercambios pueden realizarse con tres cartas del mismo tipo, una de cada tipo de ejército o dos cartas de ejército acompañadas de un comodín.

Los primeros valores de intercambio son 4, 6, 8, 10, 12 y 15 unidades. Los siguientes aumentan de cinco en cinco, según las reglas del proyecto [1].

Proceso. El jugador puede realizar intercambios mientras tenga las cartas necesarias. Cuando una carta canjeada corresponde a un territorio propio, se agregan dos unidades directamente en ese territorio.

Después se distribuyen las unidades pendientes. Cuando el reparto termina correctamente, los cambios se registran y el jugador avanza a la etapa de ataque.

Resultados. Puede retornar `COMANDO_CORRECTO`, `JUEGO_NO_INICIALIZADO`, `JUEGO_TERMINADO`, `JUGADOR_NO_VALIDO`, `JUGADOR_FUERA_DE_TURNO`, `JUGADOR_YA_OBTUVO_UNIDADES` o `ENTRADA_CERRADA`.

ATACAR

Forma de uso: `atacar nombre_jugador`

Permite realizar una o varias rondas de combate desde un territorio propio hacia un territorio enemigo vecino.

Entrada. Recibe el nombre del jugador. Posteriormente solicita si desea atacar, los códigos de los territorios de origen y destino, la decisión de continuar y, cuando ocurre una conquista, la cantidad de unidades que se trasladarán.

Condiciones. La partida debe estar activa y debe corresponder el turno al jugador indicado. También debe haberse completado la etapa de obtención de unidades.

El territorio de origen debe pertenecer al atacante y conservar más de una unidad. El destino debe pertenecer a otro jugador y tener una conexión directa con el origen.

Dados. La cantidad de dados depende de las unidades disponibles en los territorios.

El atacante utiliza un dado cuando tiene dos unidades, dos dados cuando tiene tres y tres dados cuando tiene cuatro o más.

El defensor utiliza un dado cuando tiene una unidad y dos dados cuando tiene dos o más.

Los resultados se ordenan de mayor a menor y se comparan por posición. Si el resultado del atacante es mayor, el defensor pierde una unidad; en caso contrario, la pierde el atacante. Los empates favorecen al defensor [1].

Proceso. Se comprueba la configuración de los territorios antes de lanzar los dados. Si no es válida, no se ejecuta la ronda.

En una ronda válida se obtienen y comparan los resultados de los dados y se actualizan las unidades correspondientes.

Si el defensor todavía conserva unidades, el atacante puede decidir continuar.

Cuando el defensor pierde todas sus unidades, el territorio cambia de propietario y se realiza el traslado posterior a la conquista.

Durante una etapa de ataque puede entregarse como máximo una carta por las conquistas realizadas. También se comprueba si un jugador controla los 42 territorios para determinar la victoria.

Resultados. Puede retornar `COMANDO_CORRECTO` o estados de error relacionados con la partida, el jugador, el turno y la entrada.

Entre ellos se encuentran `JUGADOR_NO_VALIDO`, `JUGADOR_FUERA_DE_TURNO`, `JUGADOR_NO_HA_UBICADO_UNIDADES`, `JUGADOR_YA_TERMINO_ATAQUE` y `ENTRADA_CERRADA`.

Una configuración territorial inválida se rechaza sin lanzar los dados y permite seleccionar nuevamente los territorios.

FORTIFICAR

Forma de uso: `fortificar nombre_jugador`

Permite realizar la última etapa del turno mediante un único traslado entre territorios propios vecinos, o terminar la etapa sin trasladar unidades.

Entrada. Recibe el nombre del jugador. Durante la ejecución solicita origen, destino y cantidad, o la letra `n` para finalizar.
Condiciones. La partida debe estar activa y el jugador debe encontrarse en su turno y en la etapa de fortificación.

Los territorios deben existir, ser diferentes, pertenecer al mismo jugador y ser vecinos.

La cantidad que se desea trasladar debe ser positiva y dejar al menos una unidad en el territorio de origen.
Proceso. Se intenta realizar el traslado sobre una copia del tablero. Si cumple las condiciones, se registran los cambios.

Si el jugador decide no fortificar, el tablero conserva su estado.

En ambos casos, al finalizar correctamente, el turno pasa al siguiente jugador con territorios y vuelve a la etapa de obtención de unidades.
Resultados. En caso de éxito retorna `COMANDO_CORRECTO`. Puede informar errores de partida, jugador o turno, incluidos `JUGADOR_NO_VALIDO`, `JUGADOR_FUERA_DE_TURNO`, `JUGADOR_NO_HA_ATACADO` y `ENTRADA_CERRADA`.

ESTADO_JUEGO

Forma de uso: `estado_juego`

Consulta la información actual de la partida sin modificarla.
Entrada. No recibe parámetros.
Condiciones. La partida debe estar inicializada y no debe haber terminado.
Proceso. Muestra la cantidad de jugadores, sus nombres y colores, el jugador actual y los datos de los territorios.

Para cada territorio presenta su continente, nombre, código, propietario y cantidad de unidades.
Resultados. Cuando la consulta es válida, presenta el estado de la partida. En caso contrario, informa `JUEGO_NO_INICIALIZADO` o `JUEGO_TERMINADO`.

AYUDA

Formas de uso:

```
ayuda
ayuda nombre_comando
```

Permite consultar los comandos disponibles y conocer las instrucciones para utilizarlos.
Entrada. Puede recibir cero o un argumento.
Condiciones. No requiere una partida inicializada. El argumento, cuando existe, debe corresponder a un comando registrado.
Proceso. La ayuda general muestra los comandos organizados por categorías.

La ayuda específica busca el nombre en `TABLA_COMANDOS` y presenta la descripción, la forma de uso, un ejemplo y la cantidad de parámetros.

La ampliación de la tabla de comandos para incluir las operaciones de almacenamiento corresponde a la integración del Componente 2.
Resultados. Muestra la ayuda solicitada o informa que el nombre del comando es inválido.

SALIR

Forma de uso: `salir`
Finaliza la ejecución del programa.
Entrada. No recibe argumentos.
Condiciones. No requiere una partida inicializada.
Proceso. Interrumpe el ciclo principal de comandos y devuelve el control a la terminal.
Resultados. Finaliza la ejecución de la aplicación.

4.2. Diseño del Componente 1

El diseño se presenta mediante la tabla de funciones auxiliares de la interfaz, el procedimiento principal, los diagramas de flujo y las especificaciones de los TAD.

Siguiendo el criterio trabajado en el Taller 2 de estructuras lineales, la función `main` se representa mediante su diagrama de flujo y no se repite dentro de la tabla de funciones auxiliares [5].

4.2.1. Funciones auxiliares de la interfaz

Las funciones auxiliares de `Comandos.cpp` separan la entrada recibida, validan los comandos y presentan la ayuda del sistema.

La Tabla 1 conserva la estructura utilizada en el documento original y en el Taller 2.

Tabla 1: Funciones auxiliares de la interfaz de comandos

Función	Propósito	Datos de entrada	Resultado	Funciones que utiliza
`buscarComando`	Busca el nombre de un comando dentro de la tabla.	Nombre del comando.	Retorna su posición o -1 si no existe.	`TABLA_COMANDOS`, `size`.
`validarCantidad Parametros`	Comprueba el mínimo y máximo de argumentos permitidos.	Posición del comando y cantidad de argumentos.	Retorna verdadero o falso. Si falla, muestra el uso y ejemplo.	`TABLA_COMANDOS`.
`validarComando`	Comprueba que el comando exista y tenga una cantidad válida de argumentos.	Nombre del comando y cantidad de argumentos.	Retorna verdadero cuando la instrucción puede ejecutarse.	`buscarComando`, `validarCantidad Parametros`.
`separarEntrada`	Divide una línea de entrada en palabras, ignorando espacios vacíos.	Línea de texto.	Retorna una secuencia de cadenas con el comando y sus argumentos.	`push_back`, `size`.
`mostrarAyuda General`	Presenta los comandos disponibles por categorías.	Ninguna.	Muestra las formas de uso y ejemplos sin modificar la partida.	`TABLA_COMANDOS`, `size`.
`mostrarAyuda Comando`	Presenta la información específica de un comando.	Nombre del comando.	Muestra descripción, uso, ejemplo y parámetros, o informa que no existe.	`buscarComando`.

4.2.2. Procedimiento principal

Ubicación. La función principal corresponde actualmente a `main.cpp`. En el documento histórico de la Entrega 1 aparecía en `Entrega0_Risk.cpp`.
Entrada. Líneas de texto escritas después del indicador `$` de la consola.
Salida. Indicador de entrada, mensajes de validación, ayuda y resultados de los comandos ejecutados.
Precondiciones. Ninguna.
Poscondiciones. El programa conserva una instancia de la partida, procesa instrucciones mientras el ciclo principal continúa activo y termina cuando se solicita el comando `salir` o se cierra la entrada.

Al iniciar la ejecución se establece la generación pseudoaleatoria, se crea la partida y comienza la lectura de instrucciones.

Cada línea se separa en palabras, se valida el nombre del comando y su cantidad de argumentos y se ejecuta la operación correspondiente.

Para la Entrega 2, este procedimiento se amplió con los comandos de almacenamiento sin sustituir las operaciones originales del Componente 1.

4.2.3. Diagramas de flujo

La Figura 2 conserva el diagrama de flujo del procedimiento principal incluido en el documento de la Entrega 1.

Este diagrama constituye una referencia histórica del primer componente. La interfaz actual incorpora también el guardado en texto y la compresión mediante Huffman, cuyo funcionamiento se desarrolla en el Componente 2.

La Figura 3 representa el procesamiento original del comando `inicializar`, que leía un archivo de texto.

En la Entrega 2 se conserva el procesamiento de jugadores y territorios y se incorpora una etapa previa de descompresión cuando el archivo recibido tiene extensión `.bin`.

La Figura 4 presenta el proceso de obtención y distribución de unidades, que incluye las validaciones del turno, los canjes de cartas y la ubicación de refuerzos.

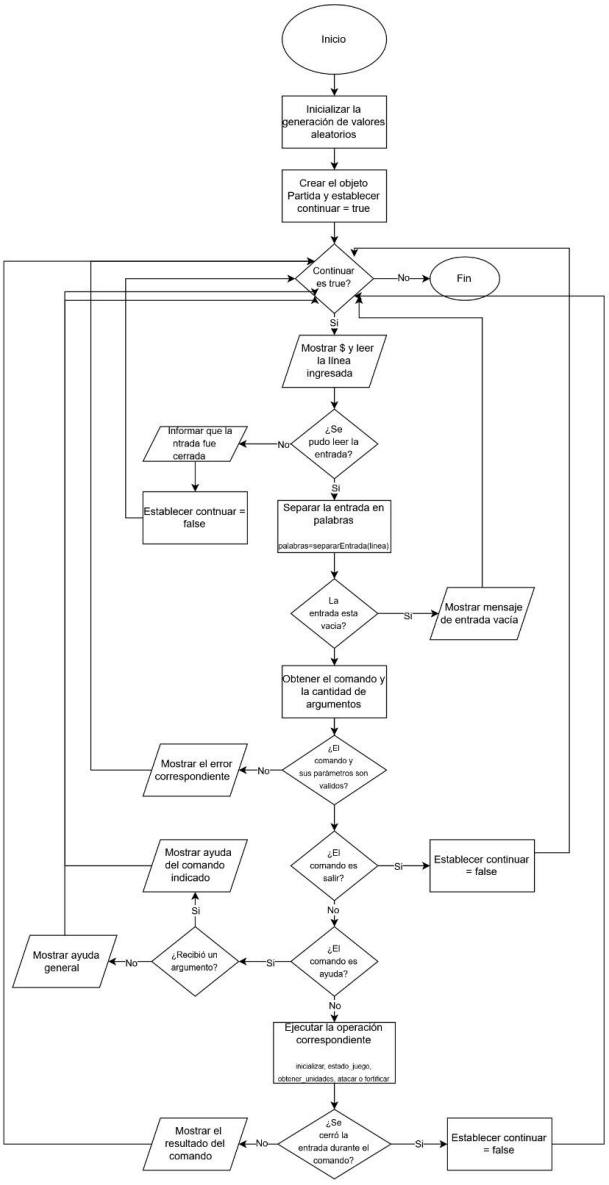


Figura 2 Diagrama de flujo de la función principal presentado en la Entrega 1. Elaboración propia del equipo.

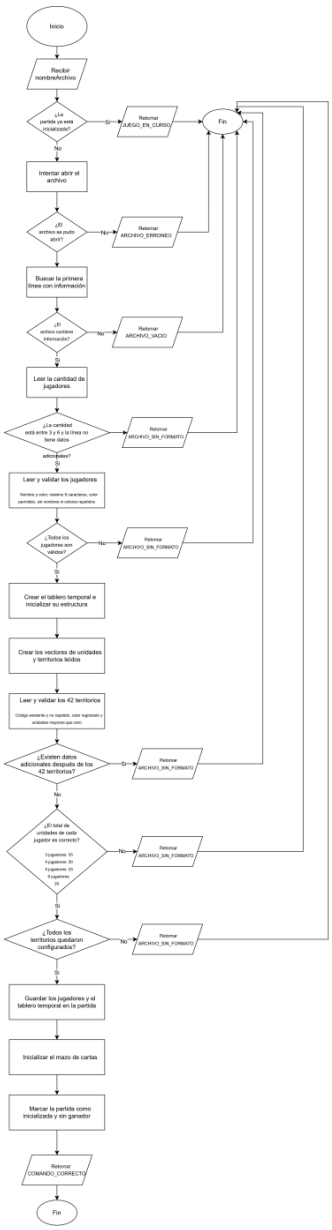


Figura 3 Diagrama de flujo original del comando inicializar presentado en la Entrega 1. Elaboración propia.

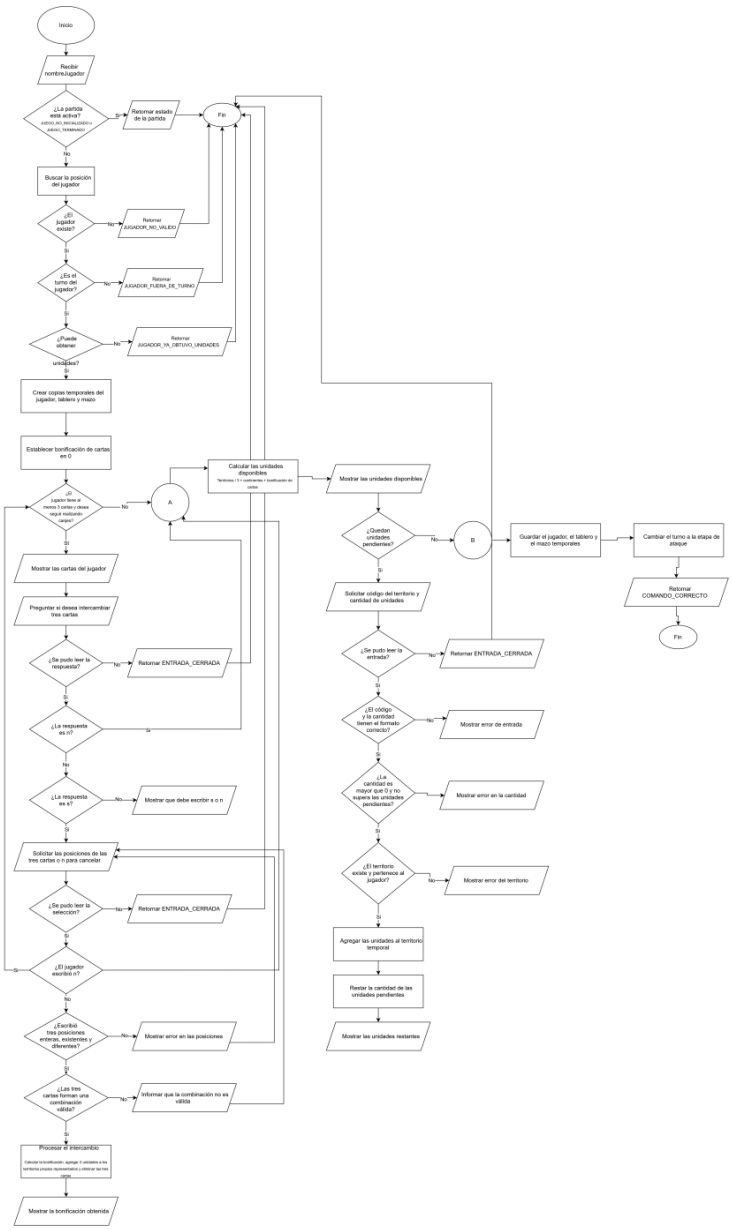


Figura 4 Diagrama de flujo del comando obtener_unidades presentado en la Entrega 1. Elaboración propia.

4.3. Decisiones y aclaraciones del Componente 1

El primer jugador escrito en el archivo de entrada ocupa la posición inicial y comienza la partida.

Durante un ataque, el territorio de origen necesita más de una unidad. La cantidad de dados disponibles disminuye cuando los territorios tienen pocas unidades.

La fortificación debe conservar al menos una unidad en el territorio de origen.

Las cartas de territorio y los comodines normales forman parte del juego obligatorio. El mazo contiene 42 cartas asociadas a territorios y dos comodines.

Las misiones secretas corresponden a una ampliación opcional y no se presentan como parte del sistema implementado hasta la Entrega 2.

En el documento de la Entrega 1 se registró una aclaración relacionada con el traslado de cero unidades después de una conquista. Esta situación corresponde al proceso de ataque y no debe confundirse con las condiciones del comando `fortificar`.

La Entrega 2 también permitió distinguir entre la distribución inicial de unidades establecida en el enunciado y la distribución existente en una partida guardada después de varios turnos.

4.4. Diseño de los TAD de la Entrega 1

Los Tipos Abstractos de Datos (TAD) utilizados en el proyecto se describen mediante su propósito, estado e interfaz.

En el estado se especifican las variables administradas, sus tipos abstractos y las condiciones que deben cumplir.

La interfaz presenta las operaciones disponibles, indicando su función, los datos recibidos, el resultado, las precondiciones y las poscondiciones.

Se conserva la organización utilizada en la documentación de la Entrega 1 y en el material de clase sobre TAD.

En este apartado se conserva el diseño de los TAD que conforman el primer componente. Las modificaciones y los nuevos TAD de la Entrega 2 se documentan en el apartado de ese componente.

4.4.1. TAD Territorio

Representa un territorio del tablero junto con su identificación, propietario, cantidad de unidades y conexiones con otros territorios.

Estado

Código del territorio (`codigo`): Cadena de caracteres.

Identifica el territorio dentro del tablero.

Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado. En un territorio del tablero debe corresponder con uno de los 42 códigos definidos.

Nombre del territorio (`nombre`): Cadena de caracteres.

Guarda el nombre con el que se presenta el territorio.

Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado.

Código del continente (`codigoContinente`): Cadena de caracteres.
Identifica el continente al que pertenece el territorio.
Condición: En un territorio configurado debe corresponder con uno de los seis continentes definidos para el tablero.

Color del propietario (`colorPropietario`): Cadena de caracteres.
Identifica al jugador que controla el territorio.
Condición: Puede estar vacío antes de cargar una partida. Después de configurar el tablero debe corresponder con el color de uno de los jugadores.

Cantidad de unidades (`unidades`): Numérico entero.
Indica cuántas unidades se encuentran en el territorio.
Condición: Su valor debe ser mayor o igual que cero. En una partida válida, los territorios ocupados deben conservar las unidades requeridas por las reglas del juego.

Códigos de los vecinos (`codigosVecinos`): Secuencia de cadenas de caracteres.
Guarda los códigos de los territorios conectados directamente con el territorio actual.
Condición: Cada código debe identificar un territorio vecino definido en el mapa.

Interfaz

`Territorio()`**: Territorio.**
Crea un territorio vacío.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un territorio con los datos de identificación y propietario vacíos, cero unidades y sin vecinos registrados.

`Territorio(nuevoCodigo, nuevoNombre, nuevoContinente)`**: Territorio.**
Crea un territorio con los datos de identificación recibidos.
Precondiciones: Los datos deben representar un territorio definido para el tablero.
Poscondiciones: Se obtiene un territorio con el código, nombre y continente recibidos. El propietario queda vacío, la cantidad de unidades es cero y no se registran vecinos.

`obtenerCodigo()`**: Cadena de caracteres.**
Consulta el código del territorio.
Precondiciones: Ninguna.
Poscondiciones: Retorna el código almacenado sin modificar el territorio.

`obtenerNombre()`**: Cadena de caracteres.**
Consulta el nombre del territorio.
Precondiciones: Ninguna.
Poscondiciones: Retorna el nombre almacenado sin modificar el territorio.

`obtenerCodigoContinente()`**: Cadena de caracteres.**
Consulta el código del continente al que pertenece el territorio.
Precondiciones: Ninguna.
Poscondiciones: Retorna el código del continente sin modificar el territorio.

`obtenerColorPropietario()`**: Cadena de caracteres.**

Consulta el color del jugador propietario.
Precondiciones: Ninguna.
Poscondiciones: Retorna el color del propietario sin modificar el territorio.

`obtenerUnidades()`: **Numérico.**
Consulta la cantidad de unidades del territorio.
Precondiciones: Ninguna.
Poscondiciones: Retorna la cantidad de unidades almacenada. El territorio conserva su estado.

`asignarPropietario(nuevoColor)`: **Vacío.**
Asigna directamente el color del propietario.
Precondiciones: El color recibido debe corresponder con el estado que se desea cargar.
Poscondiciones: El propietario queda establecido con el color recibido. Los demás datos conservan sus valores.

`asignarUnidades(nuevaCantidad)`: **Vacío.**
Asigna directamente la cantidad de unidades del territorio.
Precondiciones: La cantidad recibida debe ser mayor o igual que cero.
Poscondiciones: La cantidad de unidades queda igual al valor recibido. Los demás datos no cambian.

`agregarUnidades(cantidad)`: **Booleano.**
Intenta agregar unidades al territorio.
Precondiciones: Ninguna.
Poscondiciones: Si la cantidad recibida es positiva, se agrega a las unidades existentes y retorna verdadero. En caso contrario, retorna falso y conserva las unidades anteriores.

`retirarUnidades(cantidad, minimoRestante)`: **Booleano.**
Intenta retirar unidades conservando una cantidad mínima en el territorio.
Precondiciones: Ninguna.
Poscondiciones: Si la cantidad es positiva y después de retirarla se conserva al menos el mínimo indicado, descuenta las unidades y retorna verdadero. En caso contrario, retorna falso y conserva la cantidad anterior.

`agregarVecino(codigoVecino)`: **Vacío.**
Registra un territorio como vecino.
Precondiciones: El código recibido debe corresponder con un territorio conectado directamente.
Poscondiciones: El código queda agregado a la secuencia de vecinos.

`esVecino(codigoTerritorio)`: **Booleano.**
Comprueba si un territorio está registrado como vecino.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si el código recibido se encuentra en la secuencia de vecinos. En caso contrario, retorna falso. No modifica el territorio.

`obtenerVecinos()`: **Secuencia de cadenas de caracteres.**

Consulta los códigos de los territorios vecinos.
Precondiciones: Ninguna.
Poscondiciones: Retorna la secuencia de códigos de vecinos sin modificarla.

`cambiarPropietario(color, cantidad)`: **Booleano.**
Registra la conquista de un territorio.
Precondiciones: Ninguna.
Poscondiciones:

Si el territorio tiene cero unidades y la cantidad recibida es mayor o igual que cero, cambia el propietario, asigna la cantidad indicada y retorna verdadero.

Si el territorio aún tiene unidades o la cantidad recibida es negativa, retorna falso y conserva el estado anterior.

4.4.2. TAD Continente

Representa una región del tablero que reúne varios territorios y proporciona una bonificación de unidades cuando un jugador controla todos sus territorios.

Estado

Código del continente (`codigo`): Cadena de caracteres.
Identifica el continente dentro del tablero.
Condición: Puede estar vacío mientras el continente no haya sido configurado. Después debe corresponder con uno de los seis códigos definidos.

Nombre del continente (`nombre`): Cadena de caracteres.
Guarda el nombre visible del continente.
Condición: Puede estar vacío únicamente antes de configurar el continente.

Bonificación (`bonificacion`): Numérico entero.
Indica la cantidad de unidades adicionales que recibe un jugador al controlar completamente el continente.
Condición: En un continente configurado debe corresponder con la bonificación establecida en el enunciado. En un continente vacío es cero.

Territorios (`territorios`): Secuencia de Territorio.
Almacena los territorios que forman el continente.
Condición: Cada territorio debe tener un código de continente igual al código del continente que lo contiene.

Interfaz

`Continente()`: **Continente.**
Crea un continente vacío.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un continente con código y nombre vacíos, bonificación igual a cero y sin territorios.

`Continente(nuevoCodigo, nuevoNombre, nuevaBonificacion)`: **Continente.**
Crea un continente con los datos recibidos.

Precondiciones: Los valores deben corresponder con uno de los continentes definidos para el tablero.
Poscondiciones: El continente queda identificado con los datos recibidos, con su bonificación y sin territorios registrados.

`obtenerCodigo()`**: Cadena de caracteres.**
Consulta el código del continente.
Precondiciones: Ninguna.
Poscondiciones: Retorna el código almacenado sin modificar el continente.

`obtenerNombre()`**: Cadena de caracteres.**
Consulta el nombre del continente.
Precondiciones: Ninguna.
Poscondiciones: Retorna el nombre almacenado sin modificar el continente.

`obtenerBonificacion()`**: Numérico.**
Consulta la bonificación del continente.
Precondiciones: Ninguna.
Poscondiciones: Retorna la bonificación almacenada sin modificar el continente.

`agregarTerritorio(territorio)`**: Vacío.**
Agrega un territorio a la secuencia administrada por el continente.
Precondiciones: El territorio debe pertenecer al continente que lo recibe.
Poscondiciones: El territorio queda agregado al final de la secuencia.

`obtenerTerritorios()`**: Secuencia de Territorio.**
Consulta los territorios del continente.
Precondiciones: Ninguna.
Poscondiciones: Retorna la secuencia de territorios sin modificarla.

`estaControladoPor(colorJugador)`**: Booleano.**
Determina si todos los territorios del continente pertenecen a un jugador.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero cuando la secuencia no está vacía y todos los territorios pertenecen al color indicado. En caso contrario, retorna falso. No modifica el continente.

`buscarTerritorio(codigo)`**: Territorio o inexistente.**
Busca un territorio dentro del continente.
Precondiciones: Ninguna.
Poscondiciones: Si encuentra el territorio con el código recibido, retorna acceso al territorio. En caso contrario, informa que no existe. No modifica su información.

4.4.3. TAD Tablero

Representa el mapa completo de RISK mediante los continentes y territorios utilizados durante la partida.

Estado

Continentes (`continentes`): Secuencia de Continente.
Contiene las seis regiones que forman el tablero del juego.
Condición: Antes de inicializar la estructura puede estar vacía. Después de ejecutar `inicializarEstructura()`, debe contener seis continentes y un total de 42 territorios.

Interfaz

`Tablero()`**: Tablero.**
Crea un tablero vacío.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un tablero sin continentes.

`inicializarEstructura()`**: Vacío.**
Construye los continentes, territorios y conexiones del mapa.
Precondiciones: Ninguna.
Poscondiciones: Se configura el tablero con los seis continentes y los 42 territorios definidos para RISK.

`buscarTerritorio(codigo)`**: Territorio o inexistente.**
Busca un territorio en todo el tablero.
Precondiciones: Ninguna.
Poscondiciones: Si encuentra un territorio con el código recibido, retorna acceso al mismo. En caso contrario, informa que no existe.

`existeTerritorio(codigo)`**: Booleano.**
Comprueba si un código identifica un territorio del tablero.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si encuentra el territorio y falso en caso contrario. El tablero conserva su estado.

`configurarTerritorio(codigo, colorPropietario, unidades)`**: Booleano.**
Carga el propietario y las unidades de un territorio.
Precondiciones: Ninguna.
Poscondiciones: Si el territorio existe, asigna los datos recibidos y retorna verdadero. Si no existe, retorna falso y conserva el tablero.

`sonVecinos(codigoOrigen, codigoDestino)`**: Booleano.**
Comprueba si dos territorios tienen una conexión directa.
Precondiciones: Ninguna.
Poscondiciones: Si ambos territorios existen, comprueba si el destino está registrado entre los vecinos del origen. Si alguno no existe, retorna falso. El tablero no se modifica.

`contarTerritorios(colorJugador)`**: Numérico.**
Cuenta los territorios que pertenecen al jugador indicado.
Precondiciones: Ninguna.
Poscondiciones: Retorna la cantidad de territorios cuyo propietario corresponde con el color indicado. El tablero no se modifica.

`sumarUnidades(colorJugador)`**: Numérico.**

Suma las unidades de todos los territorios pertenecientes a un jugador.
Precondiciones: Ninguna.
Poscondiciones: Retorna la suma de unidades de los territorios cuyo propietario corresponde con el color recibido. El tablero no se modifica.

`calcularBonificacionContinentes(colorJugador)`: **Numérico.**
Calcula las unidades adicionales que recibe un jugador por controlar continentes completos.
Precondiciones: Ninguna.
Poscondiciones: Retorna la suma de las bonificaciones de los continentes controlados por el jugador indicado. El tablero no se modifica.

`agregarUnidades(codigoTerritorio, colorJugador, cantidad)`: **Booleano.**
Intenta agregar unidades a un territorio perteneciente al jugador indicado.
Precondiciones: Ninguna.
Poscondiciones: Si el territorio existe, pertenece al jugador y la cantidad es positiva, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

`trasladarUnidades(codigoOrigen, codigoDestino, colorJugador, cantidad, minimoRestante)`: **Booleano.**
Intenta trasladar unidades entre dos territorios propios vecinos.
Precondiciones: Ninguna.
Poscondiciones: Si ambos territorios existen, son vecinos, pertenecen al jugador y el origen puede conservar el mínimo indicado, se descuentan las unidades del origen, se agregan al destino y retorna verdadero. En caso contrario, retorna falso y conserva las unidades anteriores.

`cambiarPropietario(codigoTerritorio, nuevoColor, cantidad)`: **Booleano.**
Intenta registrar la conquista de un territorio.
Precondiciones: Ninguna.
Poscondiciones:

Si el territorio existe, pertenece a otro jugador, tiene cero unidades y la cantidad recibida es mayor o igual que cero, cambia el propietario, asigna la cantidad indicada y retorna verdadero.

En caso contrario, retorna falso y conserva el estado anterior.

`todosLosTerritoriosConfigurados()`: **Booleano.**
Comprueba si todos los territorios del tablero tienen propietario.
Precondiciones: El tablero debe contener los continentes y territorios del juego.
Poscondiciones: Retorna verdadero si todos los territorios tienen propietario y falso si encuentra alguno sin configurar. El tablero no se modifica.

`mostrarTablero()`: **Vacío.**
Presenta los continentes y territorios del tablero.
Precondiciones: Ninguna.
Poscondiciones: Muestra la identificación, propietario y unidades de los territorios. El tablero conserva su estado.

4.4.4. TAD Carta

Representa una carta de territorio o un comodín utilizado durante los intercambios del juego.

Estado

Tipo de carta (`tipo`): TipoCarta.
Identifica la figura o categoría de la carta.
Condición: Debe corresponder con infantería, caballería, artillería o comodín.

Código del territorio (`codigoTerritorio`): Cadena de caracteres.
Identifica el territorio asociado con la carta.
Condición: Cuando la carta no es un comodín, debe corresponder con uno de los territorios del tablero. Para un comodín debe permanecer vacío.

Interfaz

`Carta()`**: Carta.**
Crea una carta comodín.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene una carta de tipo comodín sin territorio asociado.

`Carta(nuevoTipo, nuevoCodigoTerritorio)`**: Carta.**
Crea una carta con el tipo y el código de territorio recibidos.
Precondiciones: El tipo recibido debe corresponder con un tipo de carta válido. Si es un comodín, el código debe estar vacío. En los demás casos debe identificar un territorio del tablero.
Poscondiciones: Se obtiene una carta con el tipo y el código recibidos.

`obtenerTipo()`**: TipoCarta.**
Consulta el tipo de la carta.
Precondiciones: Ninguna.
Poscondiciones: Retorna el tipo almacenado sin modificar la carta.

`obtenerCodigoTerritorio()`**: Cadena de caracteres.**
Consulta el territorio asociado con la carta.
Precondiciones: Ninguna.
Poscondiciones: Retorna el código del territorio almacenado sin modificar la carta.

`esComodin()`**: Booleano.**
Comprueba si la carta corresponde a un comodín.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si el tipo de la carta corresponde con comodín y falso en caso contrario. La carta conserva su estado.

4.4.5. TAD Jugador

Representa a uno de los participantes de la partida, identificado mediante su nombre y color, y administra las cartas que ha recibido.

Estado

Nombre del jugador (`nombre`): Cadena de caracteres.
Identifica al jugador dentro de la partida.
Condición: Al cargar una partida debe contener máximo ocho caracteres y no puede repetirse entre los jugadores.

Color del jugador (`color`): Cadena de caracteres.
Identifica los territorios que controla el jugador.
Condición: Debe corresponder con uno de los colores permitidos y no puede repetirse dentro de la partida.

Cartas del jugador (`cartas`): Secuencia de Carta.
Contiene las cartas obtenidas durante la partida.
Condición: Cada carta debe corresponder con una carta retirada del mazo y entregada al jugador.

Interfaz

`Jugador()`**: Jugador.**
Crea un jugador vacío.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un jugador con nombre y color vacíos y sin cartas almacenadas.

`Jugador(nuevoNombre, nuevoColor)`**: Jugador.**
Crea un jugador identificado mediante un nombre y un color.
Precondiciones: El nombre y el color deben cumplir las condiciones definidas para los jugadores del proyecto.
Poscondiciones: Se obtiene un jugador con el nombre y color recibidos. La secuencia de cartas queda vacía.

`obtenerNombre()`**: Cadena de caracteres.**
Consulta el nombre del jugador.
Precondiciones: Ninguna.
Poscondiciones: Retorna el nombre almacenado sin modificar el jugador.

`obtenerColor()`**: Cadena de caracteres.**
Consulta el color del jugador.
Precondiciones: Ninguna.
Poscondiciones: Retorna el color almacenado sin modificar el jugador.

`agregarCarta(nuevaCarta)`**: Vacío.**
Agrega una carta a la secuencia de cartas del jugador.
Precondiciones: La carta recibida debe ser válida.
Poscondiciones: La carta queda agregada al final de la secuencia de cartas del jugador. El nombre y el color no cambian.

`obtenerCartas()`**: Secuencia de Carta.**
Permite consultar las cartas del jugador.
Precondiciones: Ninguna.

Poscondiciones: Retorna la secuencia de cartas. La operación de consulta no modifica al jugador.

4.4.6. TAD Mazo

Administra las cartas disponibles para entregar a los jugadores y controla la cantidad de intercambios realizados durante la partida.

El mazo normal contiene 42 cartas asociadas a los territorios del tablero y dos comodines.

Las cartas de misión secreta pertenecen a una ampliación opcional y no forman parte de este TAD en la implementación actual.

Estado

Cartas disponibles (`cartasDisponibles`): Secuencia de Carta.

Contiene las cartas que todavía no han sido entregadas.

Condición: Después de inicializar el mazo, puede contener como máximo 44 cartas, correspondientes a 42 territorios y dos comodines.

Cantidad de intercambios (`cantidadIntercambios`): Numérico entero.

Cuenta los intercambios realizados por todos los jugadores de la partida.

Condición: Su valor debe ser mayor o igual que cero.

Interfaz

`Mazo()`**: Mazo.**

Crea un mazo vacío.

Precondiciones: Ninguna.

Poscondiciones: Se obtiene un mazo sin cartas disponibles y con cero intercambios.

`inicializarMazo()`**: Vacío.**

Crea las cartas utilizadas durante la partida.

Precondiciones: Ninguna.

Poscondiciones: La secuencia de cartas disponibles queda formada por 42 cartas de territorio y dos comodines. La cantidad de intercambios queda en cero.

`tieneCartasDisponibles()`**: Booleano.**

Comprueba si el mazo todavía puede entregar una carta.

Precondiciones: Ninguna.

Poscondiciones: Retorna verdadero si existe al menos una carta disponible y falso cuando el mazo está vacío. No modifica el mazo.

`entregarCarta(jugador)`**: Booleano.**

Intenta entregar una carta al jugador recibido.

Precondiciones: Ninguna.

Poscondiciones: Si existen cartas disponibles, selecciona una, la agrega al jugador, la elimina del mazo y retorna verdadero.

Si no quedan cartas disponibles, retorna falso y conserva el estado del mazo.

`combinacionValida(cartas)`**: Booleano.**

Comprueba si tres cartas forman una combinación permitida para realizar un intercambio.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si la secuencia contiene exactamente tres cartas y cumple alguna de las siguientes combinaciones:

- Tres cartas del mismo tipo de ejército.
- Una carta de cada tipo de ejército.
- Dos cartas de ejército y un comodín.

En los demás casos retorna falso. Las cartas no se modifican.

`procesarIntercambio(jugador, tablero, posicionUno, posicionDos, posicionTres)`**: Numérico.**
Valida la selección de tres cartas y realiza su intercambio.
Precondiciones: Ninguna.
Poscondiciones: Si las posiciones existen, son diferentes y las cartas forman una combinación válida, la operación realiza el intercambio.
En ese caso:

- Aumenta la cantidad de intercambios realizados.
- Calcula la bonificación correspondiente al intercambio.
- Agrega dos unidades en los territorios propios asociados con las cartas intercambiadas, cuando corresponde.
- Elimina las tres cartas seleccionadas de la secuencia de cartas del jugador.
- Retorna la cantidad de unidades obtenidas como bonificación.

Si las posiciones no son válidas o las cartas no forman una combinación permitida, retorna cero y conserva el estado anterior.

Relación entre los TAD

El TAD Carta representa individualmente cada carta utilizada durante el juego.
El TAD Jugador conserva una secuencia con las cartas que ha recibido.
El TAD Mazo administra las cartas disponibles y permite entregarlas a los jugadores.
Durante un intercambio, Mazo consulta las cartas del jugador, verifica la combinación y calcula la bonificación correspondiente.
Cuando una carta intercambiada corresponde a un territorio propio, el proceso también utiliza el TAD Tablero para agregar las unidades correspondientes.
Estas relaciones se representan en la Figura 5, que presenta los TAD principales del Componente 1 y sus relaciones de composición.

4.4.7. TAD Dados

Representa los resultados de los dados utilizados durante las rondas de combate del juego.
Permite generar los valores de un lanzamiento, ordenarlos y consultar los resultados que se utilizarán durante las comparaciones del ataque.

Estado

Resultados (`resultados`): Secuencia de Numéricos enteros.

Almacena los valores obtenidos en el último lanzamiento de dados.

Condición: Cada resultado debe encontrarse entre uno y seis.

Después de lanzar los dados, los resultados deben permanecer ordenados de mayor a menor.

Interfaz

`Dados()`**: Dados.**

Crea un conjunto de dados sin resultados almacenados.

Precondiciones: Ninguna.

Poscondiciones: Se obtiene un conjunto de dados con la secuencia de resultados vacía.

`lanzar(cantidadDados)`**: Vacío.**

Genera la cantidad solicitada de resultados y los ordena de mayor a menor.

Precondiciones: Ninguna.

Poscondiciones: Elimina los resultados anteriores.

Si la cantidad recibida es positiva, genera esa cantidad de valores comprendidos entre uno y seis y los ordena de mayor a menor.

Si la cantidad es menor o igual que cero, la secuencia de resultados queda vacía.

`obtenerResultados()`**: Secuencia de Numéricos.**

Consulta todos los resultados del último lanzamiento.

Precondiciones: Ninguna.

Poscondiciones: Retorna la secuencia de resultados sin modificar los dados.

`obtenerMayores(cantidad)`**: Secuencia de Numéricos.**

Consulta los mayores resultados disponibles en el último lanzamiento.

Precondiciones: Ninguna.

Poscondiciones: Si la cantidad recibida es positiva, retorna los primeros resultados de la secuencia ordenada sin superar la cantidad de dados disponibles.

Si la cantidad es menor o igual que cero, retorna una secuencia vacía.

Los resultados almacenados no se modifican.

`limpiar()`**: Vacío.**

Elimina los resultados del último lanzamiento.

Precondiciones: Ninguna.

Poscondiciones: La secuencia de resultados queda vacía.

4.4.8. TAD Ataque

Controla la selección de los territorios involucrados en un ataque y las rondas de dados entre el atacante y el defensor.

Su responsabilidad consiste en comprobar las condiciones del combate, ejecutar los enfrentamientos y registrar los cambios producidos en los territorios.

Estado

Dados del atacante (`dadosAtacante`): Dados.

Conserva los resultados del último lanzamiento realizado por el atacante.
Condición: Durante una ronda válida contiene entre uno y tres resultados.

Dados del defensor (`dadosDefensor`): Dados.

Conserva los resultados del último lanzamiento realizado por el defensor.
Condición: Durante una ronda válida contiene uno o dos resultados.

Interfaz

`Ataque()`**: Ataque.**

Crea el controlador utilizado para desarrollar una etapa de ataque.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un objeto Ataque con los dados del atacante y del defensor sin resultados almacenados.

`configuracionValida(codigoOrigen, codigoDestino, colorAtacante, tablero)`**: Booleano.**

Comprueba si los territorios seleccionados permiten realizar un ataque.
Precondiciones: El tablero debe contener los territorios definidos para el juego.
Poscondiciones: Retorna verdadero cuando se cumplen todas las siguientes condiciones:

- Los territorios de origen y destino existen.
- Los territorios seleccionados son vecinos.
- El origen pertenece al jugador atacante.
- El destino pertenece a un jugador diferente.
- El territorio de origen contiene más de una unidad.

Si alguna condición no se cumple, retorna falso.
El tablero no se modifica durante esta comprobación.

`ejecutar(jugadorAtacante, tablero)`**: ResultadoAtaque.**

Solicita los territorios involucrados, realiza las rondas de dados y registra una conquista cuando corresponde.
Precondiciones: El jugador debe encontrarse en la etapa de ataque y el tablero debe estar inicializado.
Poscondiciones: Retorna un ResultadoAtaque que informa si se realizó al menos una ronda de combate y si se conquistó un territorio.

Durante la ejecución puede modificar las unidades de los territorios involucrados.

Si el defensor pierde todas sus unidades y se completa la conquista, puede modificar el propietario del territorio correspondiente.

Relación entre Ataque y Dados

El TAD Ataque contiene dos objetos del TAD Dados.
Uno conserva los resultados del atacante y el otro los del defensor.

Antes de realizar una ronda, Ataque comprueba que la configuración territorial sea válida.

Cuando corresponde ejecutar el combate, utiliza los dados permitidos por la cantidad de unidades de cada territorio.

Los resultados se encuentran ordenados de mayor a menor y se comparan por posición.

Si el resultado del atacante es mayor, el defensor pierde una unidad. En caso contrario, la pierde el atacante.

Cuando existe un empate, este favorece al defensor.

Esta relación se representa en la Figura 5, correspondiente al diagrama original de los TAD de la Entrega 1.

4.4.9. TAD Turno

Conserva la posición del jugador que tiene el turno actual y la etapa que puede ejecutar.

Permite controlar el avance ordenado de las actividades del jugador durante la partida.

Las etapas consideradas son obtención de unidades, ataque y fortificación.

Estado

Posición del jugador actual (`posicionJugadorActual`): Numérico entero.

Indica la posición del jugador que tiene el turno.

Condición: Durante una partida válida debe satisfacer:

$$0 \leq \text{posicionJugadorActual} < \text{cantidad de jugadores}$$

Etapa actual (`etapaActual`): EtapaTurno.

Indica la acción que puede ejecutar el jugador.

Condición: Debe corresponder con alguna de las tres etapas del turno:

- Obtención de unidades.
- Ataque.
- Fortificación.

Interfaz

`Turno()`: Turno.

Crea el control inicial de los turnos de la partida.

Precondiciones: Ninguna.

Poscondiciones: Se obtiene un turno con la posición del jugador actual igual a cero y la etapa de obtención de unidades.

`iniciar(primeraPosicion)`: Vacío.

Establece la posición del primer jugador que tendrá el turno.

Precondiciones: La posición recibida debe corresponder con una posición existente dentro de la secuencia de jugadores.

Poscondiciones: La posición del jugador actual queda igual a la recibida.

La etapa actual queda establecida en obtención de unidades.

`obtenerPosicionJugadorActual()`: **Numérico.**
Consulta la posición del jugador que tiene el turno.
Precondiciones: Ninguna.
Poscondiciones: Retorna la posición almacenada sin modificar el turno.

`obtenerEtapaActual()`: **EtapaTurno.**
Consulta la etapa en la que se encuentra el turno.
Precondiciones: Ninguna.
Poscondiciones: Retorna la etapa actual sin modificar el turno.

`esTurnoDe(posicionJugador)`: **Booleano.**
Comprueba si una posición corresponde al jugador actual.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si la posición recibida coincide con la posición del jugador actual.
En caso contrario, retorna falso.
El turno conserva su estado.

`puedeObtenerUnidades()`: **Booleano.**
Comprueba si el turno se encuentra en la etapa de obtención de unidades.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero únicamente cuando la etapa actual corresponde a obtención de unidades.
En caso contrario, retorna falso. El turno no se modifica.

`puedeAtacar()`: **Booleano.**
Comprueba si la etapa actual permite realizar ataques.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero únicamente cuando la etapa actual es ataque.
En caso contrario, retorna falso. El turno no se modifica.

`puedeFortificar()`: **Booleano.**
Comprueba si la etapa actual permite fortificar.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero únicamente cuando la etapa actual es fortificación.
En caso contrario, retorna falso. El turno no se modifica.

`calcularNuevasUnidades(jugador, tablero, bonificacionCartas)`: **Numérico.**
Calcula las unidades que obtiene un jugador durante el turno.
Precondiciones: La bonificación por cartas debe ser mayor o igual que cero.
Poscondiciones: Retorna la división entera de los territorios controlados por el jugador entre tres, más la bonificación por continentes y la bonificación por cartas.
El jugador, el tablero y el turno conservan su estado.

`ejecutarObtencionUnidades(jugador, tablero, codigoTerritorio, cantidad)`: **Booleano.**

Intenta agregar una parte de las unidades obtenidas a un territorio del jugador.
Precondiciones: Ninguna.
Poscondiciones: Si el turno se encuentra en la etapa de obtención de unidades, la cantidad recibida es positiva y el territorio pertenece al jugador, agrega las unidades y retorna verdadero.

En caso contrario, retorna falso y conserva el tablero.

`registrarUnidadesObtenidas()`**: Vacío.**

Registra el final de la etapa de obtención de unidades.
Precondiciones: Ninguna.
Poscondiciones: Si la etapa actual corresponde a obtención de unidades, cambia a la etapa de ataque.

En cualquier otra etapa conserva su valor.

`registrarAtaqueTerminado()`**: Vacío.**

Registra la finalización de la etapa de ataque.
Precondiciones: Ninguna.
Poscondiciones: Si la etapa actual es ataque, cambia a la etapa de fortificación.

En cualquier otra etapa conserva su valor.

`ejecutarFortificacion(jugador, tablero, codigoOrigen, codigoDestino, cantidad)`**: Booleano.**

Intenta trasladar unidades entre dos territorios durante la etapa de fortificación.
Precondiciones: Ninguna.
Poscondiciones: Si la etapa actual es fortificación, la cantidad recibida es positiva y el traslado entre los territorios es válido, modifica el tablero y retorna verdadero.

En caso contrario, retorna falso y conserva el tablero.

`finalizarTurno(cantidadJugadores)`**: Vacío.**

Avanza el control del turno al siguiente jugador.
Precondiciones: La cantidad de jugadores debe ser mayor que cero.
Poscondiciones: Aumenta circularmente la posición del jugador actual.

Después de la última posición vuelve a la posición cero.

La etapa actual queda establecida en obtención de unidades.

Relación de Turno con Jugador y Tablero

El TAD Turno conserva únicamente la posición del jugador actual y la etapa que puede ejecutar.

Para calcular las unidades que recibe un jugador, utiliza la información del jugador y del tablero.

Durante la etapa de obtención, comprueba que los territorios pertenezcan al jugador antes de agregar unidades.

Durante la fortificación, utiliza el tablero para validar y realizar los traslados.

La clase Partida coordina el avance entre etapas y los turnos de los jugadores que continúan participando en el juego.

4.4.10. TAD Partida

El TAD Partida integra los elementos principales del juego y coordina las operaciones originales para inicializar, desarrollar y consultar una partida. Las ampliaciones de almacenamiento se documentan en el Componente 2.

Administra los jugadores, el tablero, el turno, las rondas de ataque y el mazo de cartas.

El estado aquí descrito corresponde al diseño previo a la incorporación de Huffman. Las nuevas operaciones no sustituyen estos atributos.

Estado

Jugadores (`jugadores`): Secuencia de Jugador.

Contiene los participantes registrados en la partida.

Condición: Antes de inicializar, la secuencia puede estar vacía.

Después de cargar una partida válida, debe contener entre tres y seis jugadores.

Sus nombres y colores no pueden repetirse.

Tablero (`tablero`): Tablero.

Conserva la información de los continentes, territorios, propietarios y unidades.

Condición: Después de inicializar la partida, debe contener los seis continentes y los 42 territorios configurados.

Cada territorio debe pertenecer a uno de los jugadores registrados y mantener una cantidad válida de unidades.

Turno (`turno`): Turno.

Controla la posición del jugador actual y la etapa que puede ejecutar.

Condición: Durante una partida activa, la posición actual debe corresponder con un jugador que todavía conserve territorios.

La etapa debe corresponder con obtención de unidades, ataque o fortificación.

Ataque (`ataque`): Ataque.

Controla las rondas de combate entre territorios.

Condición: Sus dados deben cumplir las condiciones establecidas por el TAD Dados.

Mazo (`mazo`): Mazo.

Conserva las cartas disponibles y la cantidad de intercambios realizados durante la partida.

Condición: Después de inicializar la partida, debe contener las cartas disponibles según la configuración del mazo.

Las cartas entregadas dejan de estar disponibles en el mazo.

Partida inicializada (`inicializada`): Booleano.

Indica si se ha cargado correctamente la información necesaria para jugar.

Condición: Solo puede ser verdadero cuando los jugadores y los 42 territorios fueron validados y almacenados.

Partida terminada (`terminada`): Booleano.

Indica si existe un ganador de la partida.

Condición: Solo puede ser verdadero cuando un jugador controla los 42 territorios del tablero.

Nombre del ganador (`ganador`): Cadena de caracteres.

Guarda el nombre del jugador que ganó la partida.

Condición: Permanece vacío mientras `terminada` sea falso.

Cuando la partida ha terminado, debe corresponder con el nombre de uno de los jugadores registrados.

Interfaz

`Partida()`**: Partida.**

Crea una partida sin información inicial cargada.

Precondiciones: Ninguna.

Poscondiciones: Se obtiene una partida con `inicializada` y `terminada` en falso.

El nombre del ganador queda vacío.

`inicializar(nombreArchivo)`**: EstadoComando.**

Intenta cargar la información de una partida desde un archivo.

En la Entrega 1, esta operación procesa un archivo de texto con los datos necesarios para inicializar la partida.

La ampliación para recuperar archivos comprimidos se describe en el apartado de modificaciones del TAD Partida del Componente 2.

Precondiciones: Ninguna.

Poscondiciones: Si no existe otra partida inicializada y el archivo cumple el formato esperado, registra los jugadores, configura el tablero, inicia el control del turno y prepara el mazo.

El atributo `inicializada` queda en verdadero y la operación retorna `COMANDO_CORRECTO`.

Si ocurre un error de apertura, lectura o validación, retorna el estado correspondiente y no acepta una partida incompleta.

`obtenerUnidades(nombreJugador)`**: EstadoComando.**

Gestiona los intercambios de cartas y la distribución de unidades del jugador.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está activa, el jugador existe, tiene el turno y se encuentra en la etapa de obtención de unidades, permite realizar intercambios y ubicar las unidades recibidas en territorios propios.

Cuando se completa correctamente el procedimiento, registra los cambios y avanza a la etapa de ataque.

Retorna `COMANDO_CORRECTO`.

Si alguna condición no se cumple, retorna el estado de error correspondiente.

`atacar(nombreJugador)`**: EstadoComando.**

Controla la etapa de ataque del jugador indicado.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está activa, el jugador tiene el turno y se encuentra en la etapa de ataque, permite realizar rondas de combate.

Las rondas válidas pueden modificar las unidades de los territorios.

Cuando ocurre una conquista, puede cambiar el propietario del territorio y entregar como máximo una carta durante la etapa correspondiente.

También puede registrar la finalización de la partida cuando un jugador controla los 42 territorios.

Si el jugador termina la etapa sin ganar, el turno avanza a fortificación.

La operación retorna el estado correspondiente según el resultado de la ejecución.

`fortificar(nombreJugador)`: **EstadoComando.**

Permite realizar un traslado opcional de unidades y terminar el turno del jugador.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está activa y el jugador se encuentra en la etapa de fortificación, permite realizar un traslado válido entre territorios propios vecinos o finalizar sin trasladar unidades.

Después avanza al siguiente jugador que todavía conserva territorios y establece la etapa de obtención de unidades.

Retorna el estado correspondiente según la ejecución.

`validarEstadoJuego()`: **EstadoComando.**

Comprueba si el estado actual permite utilizar la partida.

Precondiciones: Ninguna.

Poscondiciones: Retorna `JUEGO_NO_INICIALIZADO` si la partida no se ha cargado.

Retorna `JUEGO_TERMINADO` si ya existe un ganador.

En caso contrario, retorna `COMANDO_CORRECTO`.

La partida conserva su estado.

`buscarPosicionJugador(nombreJugador)`: **Numérico entero.**

Busca un jugador mediante su nombre dentro de la secuencia de participantes.

Precondiciones: Ninguna.

Poscondiciones: Si encuentra un jugador con el nombre recibido, retorna su posición.

Si el jugador no existe, retorna -1.

La secuencia de jugadores no se modifica.

`comprobarGanador()`: **Booleano.**

Comprueba si algún jugador controla los 42 territorios del tablero.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está inicializada y encuentra un jugador que controla los 42 territorios, establece `terminada` en verdadero, guarda el nombre del ganador y retorna verdadero.

En caso contrario, retorna falso.

`mostrarEstadoJuego()`: **Vacío.**

Presenta la información de los jugadores, el turno actual y el tablero.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está inicializada y no ha terminado, muestra la información actual de los participantes y territorios.

La operación no modifica los datos de la partida.

`estaInicializada()`: **Booleano.**

Consulta si la partida fue inicializada correctamente.

Precondiciones: Ninguna.

Poscondiciones: Retorna el valor de `inicializada` sin modificar la partida.

`estaTerminada()`: **Booleano.**
Consulta si la partida ha terminado.
Precondiciones: Ninguna.
Poscondiciones: Retorna el valor de `terminada` sin modificar la partida.

`obtenerGanador()`: **Cadena de caracteres.**
Consulta el nombre del ganador.
Precondiciones: Ninguna.
Poscondiciones: Retorna el nombre almacenado en `ganador`.
Si todavía no existe un ganador, retorna la cadena vacía.
La partida no se modifica.

4.5. Diagrama original de relaciones de TAD: Entrega 1

El diagrama original de la Entrega 1 muestra los TAD empleados para representar la partida, sus estados, sus interfaces y las relaciones entre ellos.

Para interpretar dichas relaciones se distinguen:

- **Composición:** un TAD administra objetos de otro TAD como parte de su estado.
- **Uso:** una operación utiliza otro TAD sin almacenarlo necesariamente como parte de su estado.

El diagrama que conserva la Entrega 1 también incluye los atributos y las operaciones representadas por el equipo. Su contenido no se sustituyó ni se redibujó para esta versión del documento.

4.5.1. Relaciones del Componente 1

La estructura central del primer componente es Partida.

Partida administra los jugadores, el tablero, el turno, el ataque y el mazo de cartas.

A su vez, Tablero contiene continentes, y cada continente contiene territorios.

Los jugadores y el mazo administran cartas.

Por último, Ataque contiene dos objetos Dados, utilizados para representar los resultados del atacante y del defensor.

La Figura 5 presenta estas relaciones.

Relaciones de uso

Además de las composiciones anteriores, algunas operaciones utilizan otros TAD sin almacenarlos como parte de su estado.

Estas relaciones se presentan en la Tabla 2.

El TAD Territorio almacena el color de su propietario.

Por tanto, la relación entre Territorio y Jugador se establece mediante ese identificador.

No se requiere que cada territorio almacene un objeto Jugador completo.

Tabla 2 Relaciones de uso entre los TAD del Componente 1

TAD que utiliza	TAD utilizado	Propósito de la relación
Ataque	Jugador y Tablero	Validar el atacante, consultar territorios y aplicar los resultados de las rondas de combate.
Turno	Jugador y Tablero	Calcular unidades, comprobar territorios y realizar la fortificación.
Mazo	Jugador y Tablero	Entregar cartas, procesar intercambios y asignar bonificaciones territoriales.
Partida	Jugador, Tablero, Turno, Ataque y Mazo	Coordinar la inicialización y los comandos del juego.

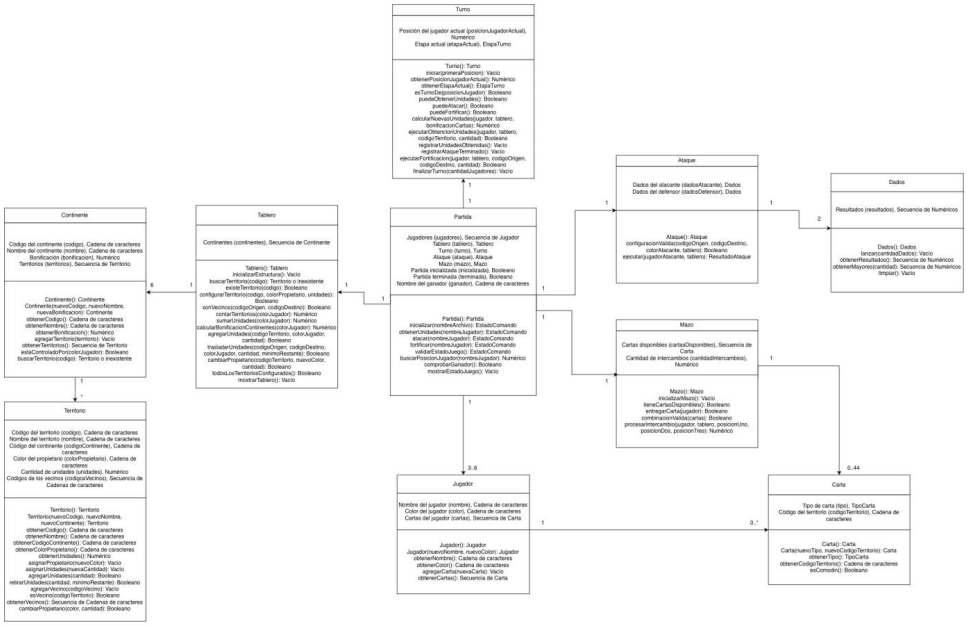


Figura 5 Diagrama original de relaciones entre los TAD del Componente 1. Se conserva como evidencia del diseño presentado en la Entrega 1; elaboración del equipo.

4.6. Plan de pruebas del Componente 1

El plan de pruebas permite comprobar que las operaciones principales de RISK produzcan los resultados establecidos por su diseño.

Se utiliza como referencia la metodología presentada en el Taller 0 de la asignatura: definir el caso de prueba, establecer los valores de entrada, determinar el resultado esperado antes de ejecutar el programa, registrar el resultado obtenido y comparar ambos resultados [2].

En este apartado se conserva el plan histórico para el comando `atacar`. El plan del comando `guardar_comprimido` se documenta separadamente dentro del Componente 2.

Los resultados históricos se distinguen de las nuevas comprobaciones para no presentar una prueba anterior como una ejecución realizada sobre una versión posterior del programa.

4.6.1. Comando atacar

Este plan fue elaborado y ejecutado durante la Entrega 1.

Su objetivo fue comprobar las validaciones previas del comando, la selección de territorios, las cantidades de unidades, las reglas de los dados y el proceso de conquista.

Se incluyeron casos normales, casos límite y casos de error.

Preparación de las pruebas

Para los casos generales se utilizó el archivo `partida.txt`, con tres jugadores y los 42 territorios del tablero.

En este archivo, Isa es la primera jugadora y se encuentra identificada con el color azul.

Para el caso límite P08 se preparó un archivo adicional llamado `prueba_origen_una_unidad.txt`.

En dicho archivo, el territorio 1.2 conserva una unidad y el territorio 1.5 tiene cinco.

La distribución fue modificada para comprobar que no se permite iniciar un ataque desde un territorio con una sola unidad.

Los dados utilizados durante el combate se generan de manera pseudoaleatoria.

Por tanto, los valores exactos de cada lanzamiento no fueron establecidos como resultado esperado antes de ejecutar el programa.

En cambio, las verificaciones se concentraron en las siguientes propiedades:

- Los resultados de los dados deben estar comprendidos entre 1 y 6.
- Los dados deben presentarse de mayor a menor.
- El número de dados debe corresponder con las unidades disponibles.
- Las pérdidas de unidades deben coincidir con la comparación de cada par de dados.
- Cuando se produce un empate, debe perder una unidad el atacante.
- La conquista debe actualizar el propietario del territorio correspondiente.

Tabla 3 Pruebas históricas del comando atacar: casos P01–P06, según el formato del Taller 0.

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
P01. Error: partida no inicializada.	Ejecutar `atacar Isa` sin inicializar.	Informar que el juego no está inicializado; no pedir territorios ni lanzar dados.	Se informó que la partida no estaba inicializada. No se pidieron territorios ni se lanzaron dados.
P02. Error: jugador inexistente.	Inicializar `partida.txt`; ejecutar `atacar Pedro`.	Rechazar al jugador inexistente sin modificar el tablero.	Se informó que Pedro no pertenecía a la partida y no se ejecutó el ataque.
P03. Error: jugador fuera de turno.	Inicializar `partida.txt`; ejecutar `atacar Saul` durante el turno de Isa.	Rechazar la solicitud e informar que el turno corresponde a Isa.	Se informó que no era el turno de Saul; el ataque no comenzó.
P04. Error: etapa incorrecta.	Inicializar `partida.txt`; ejecutar `atacar Isa` antes de `obtener_unidades`.	Indicar que Isa no ha ubicado sus unidades; no comenzar el ataque.	Se informó que Isa no había ejecutado `obtener_unidades`; el ataque no comenzó.
P05. Error: origen ajeno.	En etapa de ataque, seleccionar origen 1.1 y destino 1.2.	Rechazar porque el origen 1.1 pertenece al jugador verde; no lanzar dados.	El programa rechazó la configuración del ataque.
P06. Error: destino propio.	En etapa de ataque, seleccionar origen 1.2 y destino 1.5.	Rechazar porque ambos territorios pertenecen a Isa; no lanzar dados.	El programa rechazó la configuración del ataque.

Ejecución de las pruebas

Los casos descritos a continuación corresponden a la **ejecución histórica de la Entrega 1**, documentada por el equipo antes de incorporar Huffman. En aquella entrega se registró la compilación con `g++ -std=c++17 *.cpp` y la ejecución con `./a.out`; estos comandos no se presentan como pruebas repetidas sobre la versión de la Entrega 2.

Los casos P01–P07, P09 y P12 utilizaron `partida.txt`. Para P08 se utilizó `prueba_origen_una_unidad.txt`. Los casos P10 y P11 se desarrollaron durante la misma etapa de ataque: primero se observó un empate y después se produjo una conquista. Los resultados específicos se conservan tal como quedaron registrados en el documento original.

Planificación y resultados obtenidos

De acuerdo con el formato del Taller 0 [2], las Tablas 3 y 4 presentan para cada caso su descripción, valores de entrada, resultado esperado y resultado obtenido. No se atribuyen estas ejecuciones a la versión integrada del Componente 2.

Tabla 4 Pruebas históricas del comando atacar: casos P07–P12, según el formato del Taller 0.

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
P07. Error: territorios no vecinos.	Durante el ataque de Isa, seleccionar origen 1.2 y destino 4.1.	Rechazar porque no existe una conexión directa entre los territorios.	Se rechazó la configuración por no existir conexión directa.
P08. Límite: origen con una unidad.	Inicializar `prueba_origen_una_unidad.txt`; ubicar refuerzos en 1.5; atacar de 1.2 a 1.1.	Rechazar porque 1.2 conserva una sola unidad; no lanzar dados.	Se cargó el archivo de prueba y se rechazó el ataque antes de lanzar dados.
P09. Normal: combate sin conquista.	Ubicar cuatro unidades en 1.2; atacar 1.1 y detenerse después de una ronda.	Lanzar los dados correspondientes, actualizar las unidades y conservar el propietario si el defensor sobrevive.	Atacante: $5, 5, 1$; defensor: $4, 2$. El destino perdió dos unidades y conservó una; no hubo conquista.
P10. Límite: empate de dados.	Atacar desde 1.2 (siete unidades) hacia 1.1 (tres unidades); continuar hasta un empate.	Utilizar tres dados de ataque y dos de defensa; ante un empate pierde el atacante.	Atacante: $6, 5, 1$; defensor: $6, 1$. En el empate $6 = 6$ perdió una unidad el atacante.
P11. Conquista, traslado y carta.	Continuar el ataque 1.2 hacia 1.1 hasta conquistar; escoger cero unidades para trasladar.	Según el plan histórico: cambiar el propietario, permitir traslado de cero unidades y entregar una carta.	Atacante: $5, 3, 1$; defensor: $2, 1$. Se registró la conquista de Alaska, se aceptó cero y se entregó una carta. Véase la observación posterior.
P12. Error: territorio inexistente.	Durante el ataque de Isa, seleccionar origen 1.2 y destino 9.9.	Rechazar el código inexistente; no lanzar dados.	Se informó que el destino 9.9 no existía; el ataque no se ejecutó.

Análisis de los resultados

En el documento original de la Entrega 1, los doce casos se registraron como coincidentes entre lo esperado y lo obtenido.

Esta clasificación histórica se conserva. Sin embargo, el caso P11 requiere una aclaración: aceptar el traslado de cero unidades después de una conquista no demuestra que se cumplan las restricciones de ocupación territorial establecidas para una partida válida.

Las pruebas P01 a P04 comprobaron las validaciones previas de partida, jugador y etapa del turno.

Las pruebas P05 a P08 comprobaron configuraciones territoriales inválidas.

En estos casos, el sistema rechazó el ataque antes de ejecutar una ronda de dados.

La prueba P09 permitió comprobar un combate válido sin conquista.

En el resultado registrado, el atacante obtuvo $5, 5, 1$ y el defensor $4, 2$.

Se realizaron dos comparaciones:

$$5 > 4 \qquad \text{y} \qquad 5 > 2$$

Por tanto, el defensor perdió dos unidades, tal como se documentó.

La prueba P10 comprobó la regla del empate.

Como ambos jugadores obtuvieron un dado con valor seis, la comparación correspondiente favoreció al defensor:

$$6 = 6 \quad \Longrightarrow \quad \text{pierde el atacante}$$

La prueba P11 registró la conquista de un territorio y la entrega de una carta.

Finalmente, P12 comprobó que un código inexistente no permite iniciar un ataque.

Observación sobre el traslado posterior a una conquista

El caso P11 merece una aclaración en la revisión incremental del diseño.

En la documentación de la Entrega 1 se registró que el programa permitió trasladar cero unidades después de conquistar un territorio.

El resultado documentado indicó que Alaska cambió de propietario y quedó con cero unidades.

Este resultado se conserva como parte de la evidencia histórica de la primera entrega.

Sin embargo, debe distinguirse de las condiciones utilizadas para inicializar y validar partidas, que exigen cantidades válidas de unidades territoriales.

Al contrastar esta prueba histórica con el código de la Entrega 2, se observa que el comportamiento continúa implementado.

Ataque.cpp admite un traslado de cero unidades después de una conquista, y Territorio.cpp permite asignar esa cantidad al nuevo propietario.

Sin embargo, Partida.cpp exige que cada territorio tenga al menos una unidad al inicializar una partida desde un archivo.

Por tanto, existe una diferencia entre las condiciones aceptadas durante la conquista y las condiciones de recuperación de una partida guardada. Esta situación debe tenerse en cuenta al evaluar la persistencia del estado del juego.

Esta observación no constituye por sí misma una modificación del código ni una afirmación de que el comportamiento haya cambiado en la Entrega 2.

Registro de evidencias

Los resultados anteriores proceden del documento de diseño presentado en la Entrega 1.

Ese documento contiene las tablas de planificación y los resultados obtenidos de los doce casos.

No se presentan imágenes de terminal adicionales que no hayan sido identificadas en los materiales disponibles.

Si durante la revisión final se requieren capturas independientes de alguno de estos casos, se obtendrán ejecutando el programa y se identificarán como nuevas evidencias de comprobación.

Se evitará confundir una captura nueva con una evidencia producida durante la Entrega 1.

5. Evaluación y ajustes posteriores a la Entrega 1

La Entrega 1 se conserva como base del diseño actual. El equipo informó una calificación de 5,0 y no cuenta con observaciones textuales del docente que deban

transcribirse como correcciones obligatorias. Esta situación se consignó en el acta al comienzo del documento (Sección 1).

La nueva funcionalidad se añadió sobre el mismo sistema, sin reemplazar los TAD que administran el juego. En concreto, la Entrega 2 incorpora las operaciones de guardado en texto, codificación Huffman y recuperación de archivos binarios. El procedimiento de inicialización existente se amplía para aceptar partidas comprimidas.

Relación con el diseño anterior. Las nuevas funciones de almacenamiento afectan a `Partida` y añaden una operación de consulta textual en `Tablero`. Se explican con sus interfaces y poscondiciones dentro del Componente 2, sin volver a presentar los demás TAD como estructuras nuevas. El diagrama general de Saúl muestra el resultado de la integración.

Distinción entre corrección y ampliación. No se atribuye al profesor ninguna instrucción que no conste en las fuentes disponibles. La compatibilidad con estados de partidas guardadas es un requisito de la nueva funcionalidad; no se presenta como una observación literal de la sustentación anterior.

6. Componente 2: almacenamiento de partidas

El segundo componente del proyecto incorpora las operaciones necesarias para guardar y recuperar la información de una partida de RISK mediante archivos de texto y archivos binarios comprimidos.

El objetivo de este componente es utilizar estructuras jerárquicas para aplicar la codificación de Huffman sobre la representación textual de la partida, siguiendo los requisitos del enunciado oficial [1].

La implementación reutiliza las estructuras desarrolladas en el Componente 1. No se construye un sistema independiente: los nuevos comandos se integran en la misma interfaz de consola.

Las operaciones principales son `guardar`, `guardar_comprimido` y la versión ampliada de `inicializar`.

6.1. Representación textual de la partida

Para evitar que el almacenamiento normal y el comprimido utilicen representaciones diferentes, ambos trabajan sobre el mismo contenido textual.

La operación `obtenerTextoPartida()` construye una cadena con la información de los jugadores y territorios.

El contenido tiene la siguiente organización:

1. Cantidad de jugadores registrados en la partida.
2. Una línea por cada jugador, con su nombre y color.
3. Una línea por cada uno de los 42 territorios, indicando su código, el color del propietario y la cantidad de unidades.

El orden de los jugadores se conserva de acuerdo con el archivo utilizado para inicializar la partida.

Para obtener los registros territoriales, `Partida` utiliza la operación `obtenerTexto Territorios()`.

Esta función recorre los continentes y sus territorios y construye las líneas de salida sin modificar el tablero.

Se utiliza `ostringstream` para reunir los datos en una cadena de texto que posteriormente puede escribirse directamente o entregarse al algoritmo de Huffman [14].

El formato guarda la información de jugadores, propietarios y unidades que exige el archivo de inicialización. No constituye una copia completa de todos los datos temporales de la ejecución, como las manos de cartas o la etapa exacta de un turno en curso.

6.2. Descripción del comando guardar

GUARDAR

Forma de uso:

`guardar nombre_archivo`

Descripción.

Almacena la información actual de una partida en un archivo de texto plano, utilizando el mismo formato que procesa el comando `inicializar`.

Entrada.

Recibe el nombre o la ruta del archivo en el que se guardará la información.

Por ejemplo:

```
guardar partida_guardada.txt
```

Condiciones.

Debe existir una partida inicializada.

El contenido textual debe poder construirse correctamente y el archivo de salida debe poder abrirse para escritura.

Proceso.

La ejecución comienza en la interfaz de comandos, que identifica la instrucción y llama a `Partida::guardar()`.

Primero se comprueba que la partida haya sido inicializada.

Después se obtiene la representación textual mediante `obtenerTextoPartida()`.

Si el contenido está vacío, la operación termina con un estado de error.

Cuando existe información válida, se abre el archivo de salida utilizando `ofstream`.

El contenido se escribe mediante `write()`, conservando los caracteres de la representación textual y sin agregar un salto de línea adicional al final.

Finalmente, se cierra el archivo y se comprueba si la escritura terminó correctamente.

Resultados.

Los mensajes establecidos en el enunciado son los siguientes:

- **Juego no inicializado:** esta partida no ha sido inicializada correctamente.
- **Comando correcto:** la partida ha sido guardada correctamente.
- **Error al guardar:** la partida no ha sido guardada correctamente.

La operación no modifica los propietarios, las unidades ni los jugadores de la partida.

6.3. Descripción del comando guardar_comprimido

GUARDAR_COMPRIMIDO

Forma de uso:

`guardar_comprimido nombre_archivo`

Descripción.

Almacena el contenido textual de la partida en un archivo binario mediante el algoritmo de codificación de Huffman.

Entrada.

Recibe el nombre o la ruta del archivo binario de salida.

Por ejemplo:

```
guardar_comprimido partida.bin
```

Condiciones.

La partida debe estar inicializada.

La representación textual debe contener información y el archivo de salida debe poder abrirse y escribirse.

Proceso.

El comando invoca la operación `guardarComprimido()`.

Primero se comprueba que exista una partida inicializada.

Después se obtiene el contenido de la partida mediante `obtenerTextoPartida()`.

Si el contenido está vacío, la operación termina con un error.

En caso contrario, se llama a `comprimirHuffman()`, que realiza las siguientes tareas:

1. Calcular la frecuencia de los caracteres del contenido.
2. Construir un montículo mínimo con los nodos correspondientes a los caracteres presentes.
3. Construir el árbol binario de Huffman.
4. Generar los códigos binarios asociados a los caracteres.
5. Escribir el encabezado del archivo comprimido.
6. Agrupar los bits codificados en bytes y escribirlos.

Resultados.

Según el enunciado, los mensajes posibles son:

- **Juego no inicializado:** esta partida no ha sido inicializada correctamente.
- **Comando correcto:** la partida ha sido codificada y guardada correctamente.
- **Error al codificar y/o guardar:** la partida no ha sido codificada ni guardada correctamente.

El comando conserva el estado de la partida y genera su representación comprimida en el archivo indicado.

6.4. Codificación de Huffman

La codificación de Huffman permite representar los símbolos de un mensaje mediante códigos binarios de longitud variable.

El algoritmo considera la frecuencia con la que aparece cada símbolo. En general, los caracteres que se repiten más veces pueden representarse con códigos más cortos.

Para construir estos códigos se utiliza un árbol binario cuyas hojas representan los símbolos presentes en el mensaje.

Los nodos internos representan combinaciones de símbolos y almacenan la suma de sus frecuencias.

Este procedimiento corresponde al material trabajado en las diapositivas de Expresión y Codificación, Árboles Binarios y Montículos [6–8].

6.4.1. Conteo de frecuencias

La compresión comienza recorriendo todos los caracteres de la representación textual de la partida.

Cada carácter se interpreta como un valor de tipo `unsigned char`.

Se utiliza un arreglo de 256 posiciones en el que cada posición almacena la frecuencia de un posible símbolo.

Cada vez que aparece un carácter, se incrementa el contador correspondiente.

En este conteo se incluyen todos los caracteres presentes en el archivo, como letras, números, espacios y saltos de línea.

Los símbolos cuya frecuencia es cero no se incorporan al árbol.

Como resultado se obtiene la información necesaria para crear las hojas iniciales del árbol de Huffman.

6.4.2. Construcción del árbol de Huffman

La construcción se realiza mediante `construirArbolHuffman()`.

Para cada símbolo con frecuencia mayor que cero se crea un nodo hoja de tipo `NodoBinario<DatoHuffman>`.

Cada hoja contiene la información del símbolo y su frecuencia.

Las raíces de los nodos iniciales se insertan en `MonticuloHuffman`, una estructura que funciona como cola de prioridad mínima.

La adaptación conserva como base el montículo estudiado e implementado en los talleres de la asignatura.

La prioridad corresponde a la frecuencia más pequeña. Cuando existen frecuencias iguales, el comparador utiliza el orden de inserción como criterio adicional.

El procedimiento de construcción consiste en repetir las siguientes operaciones:

1. Extraer los dos elementos de menor frecuencia del montículo.
2. Crear un nodo padre cuya frecuencia sea la suma de las frecuencias extraídas.
3. Asociar la primera raíz extraída como hijo izquierdo del nuevo nodo.
4. Asociar la segunda raíz extraída como hijo derecho.
5. Insertar la raíz del nuevo subárbol en el montículo.

Este procedimiento continúa hasta que queda un único elemento.

Su raíz se asigna al objeto `ArbolBinario<DatoHuffman>`, que representa el árbol completo.

Los nodos que representan caracteres originales permanecen en las hojas; los nodos internos permiten reconstruir los caminos que conducen a cada símbolo.

6.4.3. Generación de códigos binarios

Una vez construido el árbol, se generan los códigos de los caracteres mediante las operaciones `generarCodigosHuffman()` y `recorrerCodigosHuffman()`.

El recorrido comienza en la raíz.

Cuando se avanza al hijo izquierdo, se agrega un bit con valor cero.

Cuando se avanza al hijo derecho, se agrega un bit con valor uno.

Al alcanzar una hoja, la secuencia acumulada desde la raíz constituye el código del símbolo almacenado.

De esta forma, los símbolos se representan mediante los caminos de un árbol binario.

Como los símbolos se encuentran en las hojas, sus códigos cumplen la propiedad de prefijo: ningún código completo es el prefijo de otro código de símbolo.

Los códigos se almacenan en un arreglo de 256 cadenas para consultarlos durante la compresión.

Cuando el mensaje contiene un único símbolo diferente, se utiliza el código `0` para representarlo.

6.4.4. Formato del archivo comprimido

El enunciado establece que el archivo comprimido debe almacenar la información necesaria para reconstruir el árbol de Huffman y recuperar el mensaje original [1].

Su organización es la siguiente:

$$n,\ (c_1, f_1),\ (c_2, f_2),\ \ldots,\ (c_n, f_n),\ w,\ \text{código binario}$$

Los campos tienen los tamaños indicados en la Tabla 5.

Tabla 5 Estructura del archivo binario de Huffman

Campo	Tamaño	Descripción
n	2 bytes	Cantidad de símbolos diferentes presentes en el mensaje original.
c_i	1 byte	Valor correspondiente al símbolo número i.
f_i	8 bytes	Frecuencia del símbolo número i.
w	8 bytes	Cantidad total de caracteres del mensaje original.
Código	Variable	Secuencia de bits de Huffman almacenada en bytes.

El encabezado contiene primero la cantidad de símbolos distintos. Después aparecen los pares de símbolo y frecuencia. Finalmente, se almacena la longitud original del mensaje.

El tamaño del encabezado, sin contar los bytes de datos comprimidos, corresponde a:

$$H = 2 + n(1 + 8) + 8$$

Por tanto:

$$\boxed{H = 10 + 9n \text{ bytes}}$$

Esta expresión permite calcular de forma independiente el tamaño esperado del encabezado para un número conocido de símbolos diferentes.

6.4.5. Escritura de los bits comprimidos

Los códigos de Huffman se concatenan siguiendo el orden de los caracteres del contenido original.

La secuencia resultante se divide en grupos de ocho bits.

Cada grupo representa un byte que se escribe en el archivo binario.

Cuando el último grupo contiene menos de ocho bits, se completa con ceros hasta alcanzar el tamaño requerido.

La longitud original w permite determinar cuántos caracteres deben recuperarse durante la descompresión y distinguir los datos reales de los bits de relleno.

La implementación comprueba que las operaciones de escritura terminen correctamente antes de informar el resultado del comando.

6.5. Recuperación de partidas comprimidas

La recuperación utiliza el mismo comando `inicializar` del Componente 1.

Cuando el nombre del archivo termina en `.bin`, la función `inicializar()` invoca `descomprimirHuffman()`.

El objetivo de esta función es recuperar la representación textual antes de procesar la información de jugadores y territorios.

6.5.1. Lectura y validación del encabezado

La descompresión comienza abriendo el archivo en modo binario.

Se lee primero el número de símbolos diferentes y después los pares de símbolo y frecuencia.

Con esta información se reconstruye el arreglo de frecuencias.

Posteriormente se lee el valor w, que corresponde a la cantidad de caracteres originales.

Se comprueba que la cantidad de símbolos sea válida, que las frecuencias sean positivas y que no se repitan símbolos en el encabezado.

También se comprueba que la suma de las frecuencias sea igual a la longitud original declarada.

Si el encabezado no cumple estas condiciones, la descompresión termina con un error.

6.5.2. Reconstrucción del árbol

Después de recuperar las frecuencias, se vuelve a ejecutar el procedimiento de construcción del árbol de Huffman.

Se crean las hojas correspondientes a los símbolos presentes.

Cada hoja se inserta en `MonticuloHuffman`.

El programa extrae sucesivamente los dos elementos de menor frecuencia, crea un nodo padre y vuelve a insertar el subárbol resultante.

Para obtener los mismos códigos empleados durante la compresión, se conserva el criterio de prioridad y desempate del montículo.

Al finalizar, se obtiene la raíz del árbol que permitirá interpretar los bits almacenados.

6.5.3. Decodificación de los caracteres

Una vez reconstruido el árbol, se leen los bytes de datos del archivo comprimido.

Cada byte se procesa desde el bit más significativo hasta el menos significativo.

El valor cero indica avanzar hacia el hijo izquierdo.

El valor uno indica avanzar hacia el hijo derecho.

Cuando se alcanza una hoja, se recupera el símbolo que contiene y se agrega al texto descomprimido.

Después se vuelve a la raíz para procesar el siguiente carácter.

El procedimiento continúa hasta recuperar exactamente los w caracteres indicados en el encabezado.

La implementación también comprueba la consistencia de las frecuencias recuperadas, los bits de relleno y la ausencia de datos adicionales que invaliden el archivo.

Si existe una inconsistencia, `descomprimirHuffman()` retorna falso.

6.5.4. Integración con inicializar

Cuando la descompresión termina correctamente, la información recuperada se almacena en una cadena de texto.

Se utiliza `istringstream` para procesar esta información como un flujo de entrada [13].

La función `inicializar` utiliza una referencia a `istream` para trabajar con archivos de texto normales o con el texto recuperado desde Huffman.

Así se reutilizan las mismas validaciones de jugadores, territorios y cantidades sin duplicar el procedimiento de lectura.

Si la descompresión falla, el sistema informa que el archivo no tiene el formato esperado.

Cuando la información es válida, se reconstruye el estado territorial de la partida a partir de los datos del archivo.

6.6. Diagramas de funcionamiento

El procedimiento de compresión se describe en los apartados anteriores y la construcción del árbol de Huffman se ilustra en el anexo correspondiente.

A continuación se conserva el diagrama de flujo de descompresión revisado por Saúl para la Entrega 2.

6.6.1. Flujo general de descompresión

La Figura 6 presenta el flujo de recuperación revisado por el equipo. Después de comprobar el encabezado, se crean las hojas del árbol y se insertan en el montículo. Mientras existe más de una raíz, se extraen los dos subárboles de menor frecuencia, se unen bajo un padre y se reinserta el nuevo subárbol. Cuando queda una raíz, comienza la decodificación de los bits.

Validaciones complementarias de la implementación. El dibujo resume las decisiones principales. El código `descomprimirHuffman()` también verifica las frecuencias reconstruidas, los bits de relleno del último byte, la ausencia de bytes adicionales y que no se supere la cantidad de caracteres declarada. Si alguna comprobación falla, retorna falso y no entrega un mensaje parcialmente recuperado. Solo después de una recuperación válida, `Partida::inicializar()` procesa los jugadores y territorios del texto resultante.

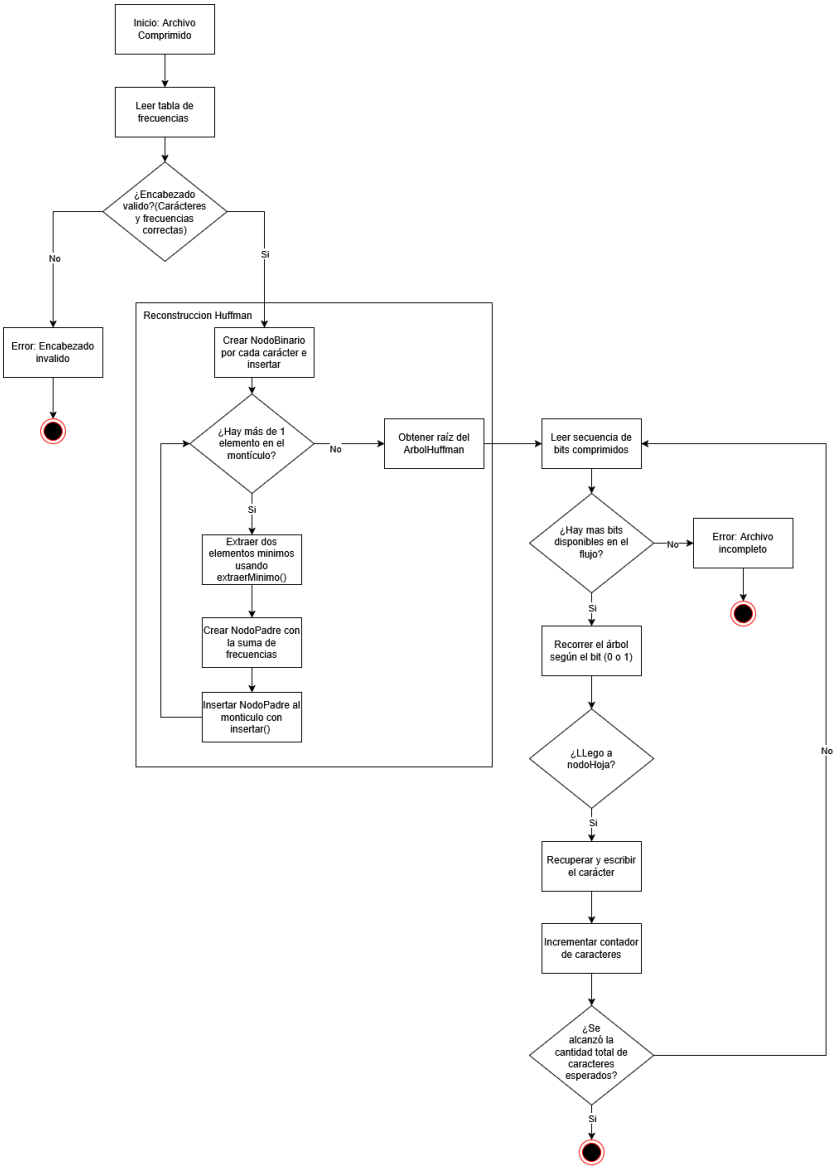


Figura 6 Diagrama de flujo de la descompresión de Huffman actualizado por el equipo para la Entrega 2.

6.7. Decisiones de diseño del Componente 2

La implementación utiliza una única representación textual para las dos operaciones de almacenamiento.

Esta decisión evita repetir el recorrido de jugadores y territorios en las operaciones `guardar` y `guardarComprimido`.

El algoritmo de Huffman se implementa mediante funciones definidas en `Huffman.cpp` y `Huffman.h`.

Las estructuras jerárquicas utilizadas son `ArbolBinario<T>` y `NodoBinario<T>`.

La selección de las raíces con menor frecuencia se realiza mediante `MonticuloHuffman`.

Se reutilizaron y adaptaron las implementaciones de árboles y montículos trabajadas en la asignatura, en lugar de introducir una biblioteca externa que sustituyera estas estructuras.

Finalmente, la recuperación de un archivo comprimido reutiliza las validaciones del comando `inicializar`.

De esta manera, las operaciones del segundo componente mantienen las responsabilidades del sistema original y agregan la codificación sin sustituir el funcionamiento del Componente 1.

6.8. Modificaciones a TAD existentes en la Entrega 2

Este apartado registra solamente las ampliaciones de las estructuras ya presentadas en la Entrega 1, siguiendo el enfoque incremental del documento de referencia de la asignatura. No se redefine el estado completo de los TAD cuando se incorporan operaciones adicionales.

6.8.1. Cambios en el TAD Tablero

Ampliación de la Entrega 2

Para implementar el almacenamiento del segundo componente se incorporó una operación que permite obtener la representación textual de los territorios.

Esta función reutiliza los continentes y territorios que ya administra el tablero y evita duplicar su recorrido desde el TAD Partida.

`obtenerTextoTerritorios()`: Cadena de caracteres.

Recorre los continentes y construye una representación textual con el código, el color del propietario y la cantidad de unidades de cada territorio.

Precondiciones: El tablero contiene su estructura territorial y los datos que se desean guardar.

Poscondiciones: Retorna las líneas correspondientes a los territorios, conservando su orden y sin modificar propietarios, unidades ni conexiones.

6.8.2. Cambios en el TAD Partida

Adaptación de inicializar en la Entrega 2

En el primer componente, `inicializar(nombreArchivo)` permitía leer los datos de la partida desde un archivo de texto.

Para el segundo componente, la función también identifica archivos cuyo nombre termina en `.bin`.

Cuando recibe un archivo comprimido, invoca `descomprimirHuffman()` para recuperar el contenido textual.

Después utiliza `istringstream` para procesar el texto recuperado mediante las mismas validaciones de jugadores y territorios.

La cantidad de unidades de un jugador puede cambiar durante la partida. Por esta razón, la recuperación de un estado guardado no exige conservar la cantidad total de unidades asignada al comenzar el juego.

Se conservan las validaciones de cantidad de jugadores, colores, nombres, territorios, propietarios y unidades válidas.

Operaciones de almacenamiento incorporadas en la Entrega 2

Para integrar el segundo componente se agregaron operaciones que permiten construir una representación textual de la partida y escribirla en un archivo de texto o en uno comprimido.

Estas operaciones reutilizan el TAD Tablero y las estructuras de la partida, sin duplicar su información interna.

Operación auxiliar interna de Partida

La función siguiente se encuentra declarada como privada en `Partida.h`: se documenta para explicar la implementación, sin incluirla entre las operaciones públicas del TAD.

`obtenerTextoPartida()`**: Cadena de caracteres.**

Construye el contenido textual utilizado para almacenar la partida.

Precondiciones: Los datos de jugadores y territorios deben encontrarse disponibles.

Poscondiciones: Retorna una cadena que comienza con la cantidad de jugadores, continúa con sus nombres y colores y termina con la información de los 42 territorios.

La representación conserva el orden de los jugadores y utiliza `obtenerTexto Territorios()` para obtener las líneas territoriales.

La operación no modifica la partida.

`guardar(nombreArchivo)`**: EstadoComando.**

Guarda el contenido actual de la partida en un archivo de texto plano.

Precondiciones: Ninguna.

Poscondiciones: Si la partida está inicializada, puede obtenerse su representación textual y el archivo de salida se escribe correctamente, retorna `COMANDO_CORRECTO`.

El archivo generado contiene la cantidad de jugadores, sus nombres y colores, y la información de los territorios.

Si la partida no está inicializada, retorna `JUEGO_NO_INICIALIZADO`.

Si ocurre un problema al generar el contenido o al escribir el archivo, retorna `ARCHIVO_ERRONEO`.

La operación no modifica el estado de la partida.

`guardarComprimido(nombreArchivo)`: **EstadoComando.**

Guarda el estado textual de la partida mediante codificación de Huffman.
Precondiciones: Ninguna.
Poscondiciones: Si la partida está inicializada, se obtiene su representación textual y se comprime correctamente mediante Huffman.

El archivo binario generado contiene el encabezado exigido en el enunciado y los bytes correspondientes a los códigos de Huffman.

En caso de éxito, retorna `COMANDO_CORRECTO`.

Si la partida no está inicializada, retorna `JUEGO_NO_INICIALIZADO`.

Si la compresión o escritura falla, retorna `ARCHIVO_ERRONEO`.

La operación conserva el estado de los jugadores y territorios.

Integración de Partida con Huffman

El TAD Partida utiliza las operaciones de Huffman para almacenar y recuperar la representación textual del juego.

El proceso de almacenamiento comprimido se organiza de la siguiente forma:

1. Partida comprueba que existe una partida inicializada.
2. Partida obtiene los datos de jugadores y territorios.
3. La información se reúne mediante `obtenerTextoPartida()`.
4. Partida invoca `comprimirHuffman()` para generar el archivo binario.
5. La función de Huffman construye el árbol y escribe el encabezado y los datos comprimidos.

Para recuperar un archivo comprimido se utiliza `inicializar()`, que llama a la función de `descomprimirHuffman()`.

Cuando la descompresión termina correctamente, la representación textual obtenida se procesa mediante las validaciones habituales de la partida.

Esta integración no agrega un atributo de tipo Huffman al estado de Partida.

Las funciones de compresión y descompresión se utilizan como operaciones auxiliares definidas fuera del TAD.

Alcance del estado recuperado

El formato utilizado por los comandos de almacenamiento corresponde al archivo de inicialización solicitado en el enunciado.

Por tanto, conserva la lista de jugadores y la configuración territorial, incluyendo propietarios y unidades.

La representación almacenada no incluye todos los detalles temporales de la ejecución.

Por ejemplo, no registra las cartas que tiene cada jugador, el contenido actualizado del mazo ni la etapa exacta en que se encontraba el turno.

Al recuperar un archivo, estos elementos se preparan conforme al procedimiento de inicialización implementado.

Esta distinción es importante para explicar correctamente qué información permiten recuperar los comandos `guardar` y `guardar_comprimido`.

Responsabilidad dentro del sistema

Partida actúa como coordinador del Componente 1 y de las operaciones de almacenamiento incorporadas en el Componente 2.

Contiene los TAD Jugador, Tablero, Turno, Ataque y Mazo.

Utiliza sus operaciones para administrar la partida sin reemplazar las responsabilidades individuales de cada estructura.

El árbol binario y el montículo se emplean dentro del proceso de Huffman y no forman parte del estado persistente del TAD Partida.

6.9. TAD y estructuras incorporados en la Entrega 2

Para implementar el almacenamiento comprimido se utilizaron las estructuras jerárquicas estudiadas durante el segundo corte y documentadas en el trabajo del equipo.

La construcción del árbol de Huffman requiere administrar nodos binarios y seleccionar repetidamente los dos subárboles cuya frecuencia sea menor.

Por esta razón, la implementación utiliza un árbol binario y una cola de prioridad mínima basada en un montículo.

Se reutilizaron como base las plantillas de árboles desarrolladas a partir del material de clase [6].

En particular, el proyecto conserva la estructura general de `NodoBinario<T>` y `ArbolBinario<T>`.

Para integrar Huffman se añadieron a `ArbolBinario<T>` las operaciones necesarias para obtener y establecer una raíz construida previamente.

Por otra parte, el montículo se especializó para administrar raíces de subárboles de Huffman [7, 9].

6.9.1. TAD Árbol Binario

El TAD Árbol Binario representa una estructura jerárquica formada por nodos que pueden tener como máximo dos hijos.

La implementación corresponde a la clase plantilla `ArbolBinario<T>`, declarada en `ArbolBinario.h` y desarrollada en `ArbolBinario.hxx`.

Para la codificación de Huffman se utiliza con datos de tipo `DatoHuffman`.

En este caso, las hojas representan símbolos del archivo original y los nodos internos almacenan frecuencias acumuladas.

El árbol utilizado no es un árbol binario de búsqueda, puesto que los símbolos no se organizan según una relación de orden.

Estado

Raíz (`raiz`): Referencia a NodoBinario.

Identifica el nodo desde el cual se puede acceder a toda la estructura del árbol.
Condición: Puede ser una referencia vacía cuando el árbol no contiene nodos.

Cuando existe una raíz, sus referencias permiten recorrer los subárboles izquierdo y derecho.

La estructura del árbol debe mantener una organización válida de nodos, sin ciclos.

Interfaz

`ArbolBinario()`**: Árbol Binario.**

Crea un árbol sin nodos.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un árbol vacío, con la referencia a la raíz sin un nodo asociado.

`ArbolBinario(valor)`**: Árbol Binario.**

Crea un árbol que contiene un único nodo raíz.
Precondiciones: El valor recibido debe ser compatible con el tipo de dato administrado por el árbol.
Poscondiciones: Se obtiene un árbol cuya raíz almacena el valor recibido y no tiene hijos.

`esVacio()`**: Booleano.**

Comprueba si el árbol contiene nodos.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero cuando no existe un nodo raíz.
En caso contrario, retorna falso.
El árbol no se modifica.

`datoRaiz()`**: T.**

Consulta el dato almacenado en la raíz del árbol.
Precondiciones: El árbol no debe estar vacío.
Poscondiciones: Retorna el dato almacenado en la raíz sin modificar la estructura.

`altura()`**: Numérico entero.**

Calcula la altura del árbol.
Precondiciones: Ninguna.
Poscondiciones: Retorna -1 cuando el árbol está vacío.
Cuando el árbol contiene solamente la raíz, retorna cero.
En los demás casos retorna la longitud del camino más largo desde la raíz hasta una hoja, medida en aristas.

`tamano()`**: Numérico entero.**

Calcula la cantidad de nodos del árbol.
Precondiciones: Ninguna.
Poscondiciones: Retorna cero cuando el árbol está vacío.
En caso contrario, retorna la cantidad total de nodos, incluida la raíz.
El árbol conserva su estado.

`insertar(padre, valor)`**: Booleano.**

Intenta insertar un nodo con el valor recibido.

Precondiciones: El valor debe corresponder con el tipo administrado por el árbol.
Poscondiciones: Si el árbol está vacío, crea la raíz y retorna verdadero.

Si encuentra el nodo padre, intenta insertar el nuevo dato en su primera posición disponible, considerando primero el hijo izquierdo y después el derecho.

Si puede insertarlo, retorna verdadero.

Cuando no encuentra el padre o no existe una posición disponible, retorna falso.

`eliminar(valor)`**: Booleano.**

Elimina un nodo identificado por su valor junto con el subárbol que depende de él.
Precondiciones: Ninguna.
Poscondiciones: Si encuentra el nodo indicado, lo elimina junto con sus descendientes y retorna verdadero.

Si el dato corresponde a la raíz, elimina el árbol completo.

Si no encuentra el dato, retorna falso.

`buscar(valor)`**: Booleano.**

Comprueba si el árbol contiene un dato.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero si encuentra un nodo cuyo dato coincide con el valor recibido.

En caso contrario, retorna falso.

El árbol conserva su estructura.

`preOrden()`**: Vacío.**

Realiza un recorrido en preorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta los datos visitando primero el nodo actual, después el subárbol izquierdo y finalmente el derecho.

No modifica el árbol.

`inOrden()`**: Vacío.**

Realiza un recorrido en inorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta los datos visitando primero el subárbol izquierdo, después el nodo actual y finalmente el derecho.

No modifica el árbol.

`posOrden()`**: Vacío.**

Realiza un recorrido en posorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta los datos visitando primero el subárbol izquierdo, después el derecho y finalmente el nodo actual.

No modifica el árbol.

`nivelOrden()`**: Vacío.**

Realiza un recorrido por niveles.
Precondiciones: Ninguna.
Poscondiciones: Presenta los nodos comenzando por la raíz y continuando nivel por nivel.

Utiliza una cola para organizar temporalmente los nodos pendientes.

El árbol no se modifica.

Operaciones incorporadas para el Componente 2

Para reutilizar el árbol binario en la construcción de Huffman se incorporaron dos operaciones.

Estas permiten trabajar directamente con la raíz del árbol, sin cambiar el funcionamiento de las operaciones generales de la plantilla.

`obtenerRaiz()`**: Referencia a NodoBinario.**

Permite consultar la raíz del árbol.

Precondiciones: Ninguna.

Poscondiciones: Retorna la referencia al nodo raíz.

Si el árbol está vacío, retorna una referencia vacía.

No crea ni elimina nodos.

`fijarRaiz(nuevaRaiz)`**: Booleano.**

Intenta establecer como raíz un nodo construido previamente.

Precondiciones: Ninguna.

Poscondiciones: Si el árbol se encuentra vacío y la referencia recibida no es nula, establece la nueva raíz y retorna verdadero.

Si el árbol ya contiene una raíz o la referencia es nula, retorna falso y no modifica la estructura.

El subárbol recibido debe tener un único responsable de su liberación para evitar problemas de memoria.

Uso en Huffman

Durante la construcción de Huffman, los nodos y subárboles se crean y combinan mediante la cola de prioridad.

Cuando queda una única raíz, esta se establece en el `ArbolBinario <DatoHuffman>`.

La operación `fijarRaiz()` permite realizar esta asignación sin tener que reconstruir el árbol mediante inserciones tradicionales.

La operación `obtenerRaiz()` permite iniciar los recorridos necesarios para generar los códigos y recuperar los caracteres.

6.9.2. Estructura auxiliar NodoBinario

La clase plantilla `NodoBinario<T>` representa cada uno de los nodos que forman el árbol binario.

Se declara en `NodoBinario.h` y se desarrolla en `NodoBinario.hxx`.

Su implementación conserva la estructura trabajada en los talleres de árboles.

Aunque es una clase con operaciones propias, en este componente cumple la función de soporte del TAD Árbol Binario.

Estado

Dato (`dato`): T.
Almacena la información asociada al nodo.
Condición: Cuando se utiliza en Huffman, el tipo T corresponde a `DatoHuffman`.

Hijo izquierdo (`hijoIzq`): Referencia a NodoBinario.
Identifica la raíz del subárbol izquierdo.
Condición: Puede estar vacía cuando el nodo no tiene hijo izquierdo.

Hijo derecho (`hijoDer`): Referencia a NodoBinario.
Identifica la raíz del subárbol derecho.
Condición: Puede estar vacía cuando el nodo no tiene hijo derecho.

Interfaz

`NodoBinario()`**: Nodo Binario.**
Crea un nodo sin hijos.
Precondiciones: Ninguna.
Poscondiciones: Se obtiene un nodo con ambas referencias a hijos vacías.

`NodoBinario(valor)`**: Nodo Binario.**
Crea un nodo con un dato inicial.
Precondiciones: El valor debe corresponder con el tipo almacenado.
Poscondiciones: El nodo contiene el valor recibido y no tiene hijos.

`obtenerDato()`**: T.**
Consulta el dato almacenado.
Precondiciones: El nodo debe estar correctamente construido.
Poscondiciones: Retorna el dato del nodo sin modificarlo.

`fijarDato(valor)`**: Vacío.**
Reemplaza el dato almacenado.
Precondiciones: El valor debe ser compatible con el tipo T.
Poscondiciones: El nodo conserva el nuevo dato recibido. Sus hijos no cambian.

`obtenerHijoIzq()`**: Referencia a NodoBinario.**
Consulta el hijo izquierdo.
Precondiciones: Ninguna.
Poscondiciones: Retorna la referencia al hijo izquierdo o una referencia vacía si no existe.

`obtenerHijoDer()`**: Referencia a NodoBinario.**
Consulta el hijo derecho.
Precondiciones: Ninguna.
Poscondiciones: Retorna la referencia al hijo derecho o una referencia vacía si no existe.

`fijarHijoIzq(nodo)`**: Vacío.**
Establece la referencia al hijo izquierdo.
Precondiciones: La referencia debe identificar un subárbol válido o ser vacía.

No debe introducir ciclos ni asignar simultáneamente la propiedad del mismo subárbol a dos padres.
Poscondiciones: El hijo izquierdo queda asociado con la referencia recibida.
El hijo derecho y el dato del nodo se conservan.

`fijarHijoDer(nodo)`**: Vacío.**
Establece la referencia al hijo derecho.
Precondiciones: La referencia debe identificar un subárbol válido o ser vacía.
No debe introducir ciclos ni compartir la propiedad de un mismo subárbol con otro padre.
Poscondiciones: El hijo derecho queda asociado con la referencia recibida.
El hijo izquierdo y el dato del nodo se conservan.

`esHoja()`**: Booleano.**
Determina si el nodo carece de hijos.
Precondiciones: Ninguna.
Poscondiciones: Retorna verdadero cuando las referencias a ambos hijos están vacías.
En caso contrario, retorna falso.
El nodo no se modifica.

`buscarNodo(valor)`**: Referencia a NodoBinario.**
Busca un dato dentro del subárbol cuya raíz es el nodo actual.
Precondiciones: El tipo del dato permite comparar valores.
Poscondiciones: Retorna una referencia al nodo encontrado.
Si no existe un dato coincidente, retorna una referencia vacía.
No modifica el subárbol.

`eliminarNodo(valor)`**: Booleano.**
Intenta eliminar un descendiente identificado por su valor.
Precondiciones: El tipo del dato permite comparar valores.
Poscondiciones: Si encuentra un descendiente con el dato recibido, lo elimina junto con su subárbol y retorna verdadero.
En caso contrario, retorna falso.
La eliminación del propio nodo raíz corresponde al objeto que administra esa raíz.

`altura()`**: Numérico entero.**
Calcula la altura del subárbol.
Precondiciones: Ninguna.
Poscondiciones: Retorna cero si el nodo es una hoja.
En caso contrario, retorna uno más la mayor altura de sus hijos existentes.

`tamano()`**: Numérico entero.**
Calcula la cantidad de nodos del subárbol.
Precondiciones: Ninguna.
Poscondiciones: Retorna la cantidad total de nodos del subárbol, incluyendo el nodo actual.

`preOrden()`**: Vacío.**

Recorre el subárbol en preorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta primero el dato del nodo actual, después el subárbol izquierdo y finalmente el derecho.

`inOrden()`**: Vacío.**
Recorre el subárbol en inorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta primero el subárbol izquierdo, después el dato actual y finalmente el subárbol derecho.

`posOrden()`**: Vacío.**
Recorre el subárbol en posorden.
Precondiciones: Ninguna.
Poscondiciones: Presenta primero el subárbol izquierdo, después el derecho y finalmente el dato del nodo actual.

Administración de los nodos

Las operaciones `fijarHijoIzq()` y `fijarHijoDer()` asignan directamente las referencias recibidas.

No comprueban por sí mismas si se forman ciclos o si un mismo subárbol queda asociado con varios padres.

Por tanto, la construcción de Huffman debe garantizar estas condiciones antes de establecer los hijos.

El destructor de `NodoBinario<T>` libera recursivamente los subárboles que dependen de sus hijos.

Por esta razón, la construcción de Huffman debe mantener una organización en la que cada subárbol tenga un único propietario.

Durante la combinación, los dos nodos extraídos del montículo pasan a depender de un nuevo nodo padre.

El montículo no debe destruirlos al extraerlos, ya que siguen formando parte del árbol que se está construyendo.

6.9.3. TAD Cola de Prioridad

El TAD Cola de Prioridad administra elementos según un criterio de prioridad.

En el proyecto se utiliza una cola de prioridad mínima, implementada mediante `MonticuloHuffman`.

Su responsabilidad consiste en mantener disponibles las raíces de los subárboles que todavía deben combinarse durante la construcción de Huffman.

Estado

Elementos (`datos`): Secuencia de ElementoMonticuloHuffman.

Contiene las referencias a las raíces de los subárboles pendientes de combinarse.
Condición: Los elementos se organizan según la propiedad de un montículo mínimo.

Cada elemento debe contener una referencia válida a la raíz de un subárbol y su orden de inserción.

La prioridad se determina mediante la frecuencia del nodo raíz.

Cuando dos elementos tienen la misma frecuencia, se utiliza el orden de inserción como criterio de desempate.

El montículo no es propietario de los nodos a los que apuntan sus elementos.

Interfaz

`MonticuloHuffman()`**: Cola de Prioridad.**

Crea una cola de prioridad vacía.

Precondiciones: Ninguna.

Poscondiciones: Se obtiene un montículo sin elementos almacenados.

`insertar(raiz, orden)`**: Booleano.**

Intenta incorporar la raíz de un subárbol en la cola de prioridad.

Precondiciones: Ninguna.

Poscondiciones: Si la referencia recibida es nula, retorna falso y conserva el montículo sin modificaciones.

En caso contrario, crea un elemento que contiene la raíz y el orden de inserción recibido.

Agrega el elemento a la secuencia y reorganiza el montículo mediante `std::push_heap()`.

Finalmente, retorna verdadero.

El orden permite desempatar cuando dos subárboles tienen la misma frecuencia.

`extraerMinimo()`**: Referencia a NodoBinario.**

Obtiene y retira la raíz del subárbol con mayor prioridad de extracción, es decir, el que tiene menor frecuencia.

Precondiciones: Ninguna.

Poscondiciones: Si el montículo está vacío, retorna una referencia nula y no modifica su estado.

En caso contrario, utiliza `std::pop_heap()` para ubicar el elemento de mayor prioridad al final de la secuencia.

Retira ese elemento y retorna la referencia a la raíz del subárbol con menor frecuencia, considerando el criterio de desempate.

La raíz retornada no se destruye y la propiedad del montículo se conserva.

`esVacio()`**: Booleano.**

Comprueba si el montículo contiene elementos.

Precondiciones: Ninguna.

Poscondiciones: Retorna verdadero cuando no quedan elementos almacenados.

En caso contrario, retorna falso.

No modifica el montículo.

Implementación del montículo

La implementación utiliza una secuencia de tipo `deque` para almacenar los elementos.

Para mantener el orden por prioridad se emplean las operaciones `std::push_heap()` y `std::pop_heap()`.

La comparación se realiza mediante `ComparadorMonticuloHuffman`.

Este comparador permite obtener primero el elemento con menor frecuencia.

La implementación se especializó para los subárboles utilizados por Huffman, tomando como referencia los montículos estudiados en los talleres.

No se incorporaron operaciones de eliminación arbitraria o de recorrido que no fueran necesarias para esta construcción.

6.9.4. Registros auxiliares de Huffman

Además de los TAD anteriores, la implementación utiliza registros simples para almacenar información asociada con los símbolos y las prioridades.

Estos registros no se presentan como TAD independientes, puesto que su propósito es almacenar datos que utilizan otras estructuras.

DatoHuffman

El registro `DatoHuffman`, definido en `Tipos.h`, contiene la información de los nodos utilizados por Huffman.

Su estado está formado por dos campos:

- **Símbolo** (`simbolo`): carácter sin signo.
  Identifica el carácter representado por una hoja.
- **Frecuencia** (`frecuencia`): numérico entero sin signo de 64 bits (`unsigned long long` en la implementación).
  Almacena la frecuencia del símbolo o la suma de las frecuencias de un subárbol.

En las hojas, el símbolo corresponde con un carácter del mensaje original.

En los nodos internos, la frecuencia corresponde a la suma de las frecuencias de sus dos hijos.

Los nodos internos no representan directamente un carácter original.

ElementoMonticuloHuffman

El registro `ElementoMonticuloHuffman` representa un elemento almacenado dentro del montículo.

Contiene los siguientes campos:

- **Raíz** (`raiz`): referencia a `NodoBinario<DatoHuffman>`.
  Identifica la raíz del subárbol que participa en la construcción.
- **Orden** (`orden`): numérico entero sin signo (`unsigned long long` en la implementación).
  Registra el orden de inserción utilizado para resolver empates de frecuencia.

Este registro permite almacenar en la cola de prioridad tanto las hojas iniciales como los subárboles que se forman durante las uniones sucesivas.

ComparadorMonticuloHuffman

El comparador determina la prioridad relativa de dos elementos del montículo.

Primero compara las frecuencias de los nodos raíz.

El elemento con menor frecuencia tiene mayor prioridad de extracción.

Cuando las frecuencias son iguales, se utiliza el orden de inserción para obtener un desempate determinista.

Este criterio resulta importante porque permite construir nuevamente el mismo árbol durante la descompresión.

Para revisar las operaciones sobre el montículo se consultó la documentación técnica de `push_heap` y `pop_heap` [15, 16].

6.9.5. Módulo de codificación Huffman

El archivo `Huffman.cpp` contiene las funciones necesarias para construir el árbol, generar códigos binarios, comprimir y descomprimir la información.

Estas funciones no constituyen por sí mismas un nuevo TAD, ya que no administran un estado persistente propio de un objeto.

Durante cada operación se crean y utilizan temporalmente las frecuencias, los nodos, el montículo y el árbol.

Las funciones principales son las siguientes:

- `construirArbolHuffman()`: construye el árbol a partir de las frecuencias.
- `generarCodigosHuffman()`: obtiene los códigos correspondientes a los símbolos.
- `recorrerCodigosHuffman()`: función auxiliar interna definida en `Huffman.cpp`. Realiza el recorrido recursivo del árbol para construir los códigos binarios. No pertenece a la interfaz declarada en `Huffman.h`.
- `comprimirHuffman()`: genera el archivo binario comprimido.
- `descomprimirHuffman()`: reconstruye el árbol y recupera el contenido original.

La interacción entre estas funciones y las estructuras jerárquicas permite almacenar y recuperar las partidas comprimidas.

La adaptación de la lectura y escritura binaria se contrastó con la documentación de los modos de apertura y las operaciones `write()` y `read()` de C++ [10–12].

6.9.6. Relación entre las estructuras del Componente 2

El TAD Árbol Binario administra la raíz del árbol completo.

La clase NodoBinario representa los nodos y permite relacionar cada padre con sus hijos izquierdo y derecho.

DatoHuffman proporciona el símbolo y la frecuencia almacenados en cada nodo.

El TAD Cola de Prioridad, implementado mediante MonticuloHuffman, administra temporalmente las raíces de los subárboles pendientes.

Cada elemento de esta cola contiene una referencia a un NodoBinario y un orden de inserción.

Finalmente, las funciones del módulo Huffman utilizan estas estructuras para construir el árbol, generar los códigos y recuperar los caracteres.

Las relaciones entre los TAD del juego y las estructuras utilizadas para Huffman se representan en la Figura 7.

Este diagrama muestra la integración de los componentes existentes con las estructuras jerárquicas incorporadas durante la Entrega 2.

La relación entre Partida y Huffman corresponde al uso de funciones de compresión y descompresión.

No se trata de que Partida contenga un objeto Huffman como atributo.

6.10. Diagrama actualizado de TAD y estructuras: Entrega 2

El segundo componente reutiliza Partida y agrega las estructuras necesarias para implementar la codificación de Huffman.

Partida invoca las funciones de compresión y descompresión definidas en el módulo Huffman.

Estas funciones utilizan un Árbol Binario y una Cola de Prioridad.

El árbol binario administra los nodos de la estructura jerárquica.

La cola de prioridad administra elementos que contienen referencias a las raíces de los subárboles pendientes de combinar.

La Figura 7 distingue los TAD de las estructuras auxiliares.

6.10.1. Criterios de interpretación del diagrama

Las funciones definidas en `Huffman.cpp` no constituyen una clase Huffman dentro de Partida.

Por este motivo, el diagrama representa una relación de uso y no una relación de composición entre Partida y Huffman.

Del mismo modo, MonticuloHuffman no administra directamente la propiedad de los nodos.

Cada elemento del montículo contiene una referencia a la raíz de un subárbol.

La propiedad de los nodos se conserva durante las combinaciones y finalmente queda administrada por el Árbol Binario completo.

La función de estas estructuras es temporal: se utilizan mientras se comprime o descomprime un archivo.

No forman parte del estado permanente de la clase Partida.

6.11. Resumen de las modificaciones de la Entrega 2

La Entrega 1 aportó los TAD necesarios para administrar el juego mediante estructuras lineales.

La Entrega 2 conserva estos TAD e incorpora el Árbol Binario y la Cola de Prioridad para implementar Huffman.

Las funciones nuevas de almacenamiento se integran mediante Partida y reutilizan la información del Tablero.

Por tanto, los componentes no funcionan de manera independiente, sino que forman parte del mismo sistema RISK.

Este diseño corresponde al estado conjunto de los Componentes 1 y 2, según el carácter incremental exigido para la segunda entrega.

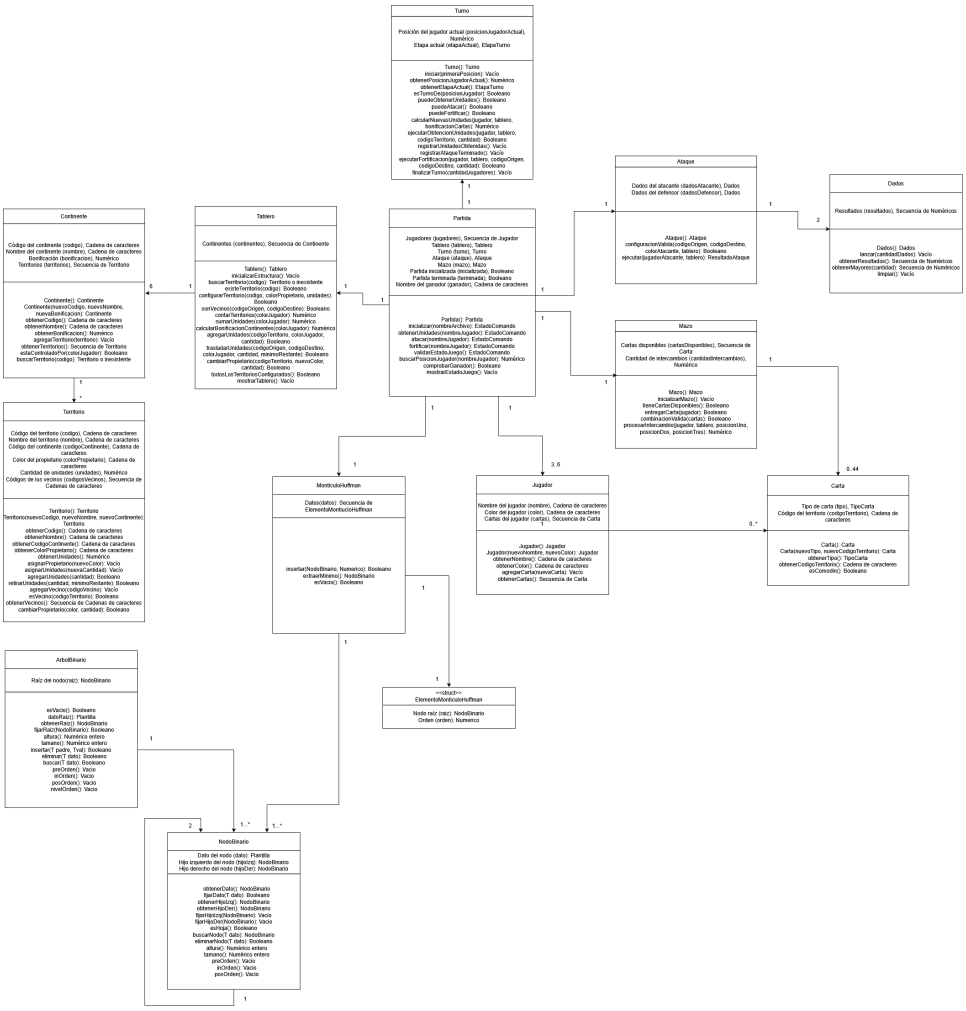


Figura 7 Diagrama general actualizado de clases y TAD para los Componentes 1 y 2. Elaboración del equipo. El recuadro Huffman representa un módulo de funciones que utiliza estructuras jerárquicas, no un atributo de tipo Huffman almacenado por Partida.

6.12. Plan de pruebas del comando guardar_comprimido

El propósito de este plan es comprobar que el comando `guardar_comprimido` almacene correctamente la representación textual de una partida utilizando la codificación de Huffman.

También se verifica que el contenido comprimido pueda recuperarse y que los errores sean identificados sin aceptar archivos inválidos.

De acuerdo con el Taller 0, el plan incluye casos normales, de error y de límite, con entradas y resultados esperados definidos antes de considerar los resultados de ejecución.

6.12.1. Objetivos de las pruebas

Se pretende comprobar lo siguiente:

1. El comando rechaza el guardado cuando no existe una partida inicializada.
2. Una partida válida puede almacenarse en un archivo binario.
3. Los datos guardados corresponden al estado territorial actual, incluyendo las modificaciones realizadas durante el juego.
4. La información comprimida corresponde a la misma representación textual utilizada por `guardar`.
5. El archivo respeta la estructura binaria definida en el enunciado.
6. El sistema informa un error cuando no puede crear o escribir el archivo solicitado.
7. El contenido comprimido puede recuperarse mediante `inicializar`.
8. Un archivo binario inválido no debe aceptarse como una partida válida.

6.12.2. Preparación y archivos utilizados

Las pruebas corresponden a la implementación en C++17 del proyecto RISK.

Se utiliza la interfaz de consola del sistema y el archivo `partida.txt`, que contiene tres jugadores y los 42 territorios.

Para las verificaciones que requieren modificar el estado del juego, se utiliza una partida inicialmente válida y se registran los cambios realizados antes del guardado.

Los archivos de prueba se organizan con nombres diferentes para evitar sobrescribir accidentalmente los archivos originales.

Se distinguen las siguientes clases de archivo:

- Archivos de texto generados mediante `guardar`.
- Archivos binarios generados mediante `guardar_comprimido`.
- Copias de archivos binarios utilizadas para comprobar el manejo de información inválida.

Los archivos empleados para las pruebas de errores deben ser copias. No se debe modificar ni sobrescribir el archivo original `partida.txt`.

6.12.3. Criterios generales de aceptación

El enunciado establece tres resultados posibles para el comando `guardar_comprimido`:

- **Juego no inicializado:** la partida no ha sido inicializada correctamente.
- **Comando correcto:** la partida ha sido codificada y guardada correctamente.
- **Error al codificar y/o guardar:** la partida no ha sido codificada ni guardada correctamente.

Una respuesta de éxito en pantalla no constituye, por sí sola, una comprobación completa.

Cuando la prueba lo requiera, también se verificará la existencia del archivo, su estructura binaria y la posibilidad de recuperar la información original.

6.12.4. Planificación de casos directos

La Tabla 6 presenta los casos que comprueban directamente el comportamiento del comando `guardar_comprimido`.

Tabla 6 Plan de pruebas directas de guardar_comprimido

ID	Tipo de caso	Preparación o entrada	Resultado esperado
GC01	Error: partida no inicializada.	Ejecutar `guardar_comprimido` con un nombre de archivo binario antes de inicializar una partida.	El sistema informa que la partida no está inicializada. No se genera un nuevo archivo comprimido.
GC02	Normal: guardado válido.	Inicializar una partida desde `partida.txt` y ejecutar `guardar_comprimido` sobre un archivo nuevo con extensión `.bin`.	Se presenta el mensaje de éxito y se crea el archivo binario correspondiente. Su contenido incluye el encabezado y la codificación exigidos.
GC03	Normal: partida modificada.	Inicializar la partida, realizar una operación válida que modifique las unidades de uno o más territorios y guardar el estado comprimido.	El archivo almacena la configuración territorial actualizada, no únicamente la distribución inicial.
GC04	Consistencia: texto y binario.	Sin modificar la partida entre operaciones, ejecutar `guardar` y `guardar_comprimido` sobre archivos diferentes.	La descompresión del binario debe recuperar exactamente el contenido generado por el guardado en texto.
GC05	Consistencia: guardado repetido.	Guardar dos veces el mismo estado de la partida en dos archivos binarios distintos, sin realizar operaciones intermedias.	Los archivos deben contener una representación equivalente. Con el criterio de desempate determinista implementado, se espera que los bytes sean idénticos.
GC06	Error: destino no disponible.	Inicializar una partida válida e intentar guardar el archivo dentro de un directorio de salida inexistente.	El programa informa un error al codificar y/o guardar. No informa que el guardado terminó correctamente.

6.12.5. Pruebas complementarias de recuperación

El comando `inicializar` debe poder recuperar los archivos generados por `guardar_comprimido`.

Por esta razón, las pruebas GC07 a GC09 verifican que los resultados del guardado puedan recuperarse y que se rechacen archivos binarios inválidos.

Estas pruebas complementan, pero no sustituyen, las pruebas directas del comando.

Tabla 7 Pruebas complementarias de recuperación de archivos Huffman

ID	Tipo de caso	Preparación o entrada	Resultado esperado
GC07	Normal: recuperación correcta.	Generar un archivo comprimido, finalizar la ejecución del programa y comenzar una nueva. Ejecutar `inicializar` con el archivo binario.	La partida se inicializa correctamente. Los jugadores, propietarios y cantidades de unidades recuperados coinciden con los datos guardados.
GC08	Error: archivo binario alterado.	Crear una copia de un archivo comprimido válido y alterar su contenido binario. Intentar inicializar una partida desde esa copia.	El archivo debe rechazarse cuando la alteración viola las comprobaciones del formato o del contenido. No debe aceptarse una partida incompleta.
GC09	Error: archivo truncado.	Crear una copia incompleta de un archivo binario válido e intentar cargarla mediante `inicializar`.	El sistema identifica que faltan datos necesarios para recuperar el contenido y rechaza el archivo. No se inicializa una partida incompleta.

6.12.6. Verificación del formato binario

Para verificar el formato binario de los archivos generados, se toma como criterio que el encabezado respete la siguiente organización:

$$n,\ (c_1, f_1), \ldots, (c_n, f_n),\ w,\ \text{código binario.}$$

El enunciado indica los siguientes tamaños:

- n: dos bytes.
- c_i: un byte por símbolo.
- f_i: ocho bytes por frecuencia.
- w: ocho bytes para la longitud original.
- Datos comprimidos: cantidad variable de bytes.

Para un archivo con n símbolos diferentes, el tamaño esperado del encabezado es:

$$H = 2 + 9n + 8$$

$$\boxed{H = 10 + 9n}$$

El valor w debe ser igual a la cantidad de bytes de la representación textual original. Asimismo, la suma de las frecuencias declaradas debe coincidir con esa longitud:

$$\sum_{i=1}^{n} f_i = w$$

Estas relaciones proporcionan resultados esperados que pueden comprobarse sin depender únicamente del mensaje de éxito del programa.

No se exige que todo archivo comprimido sea necesariamente menor que el archivo de texto.

El encabezado también ocupa espacio, por lo que el tamaño final depende del mensaje y de las frecuencias de sus símbolos.

6.12.7. Comprobación del contenido recuperado

Para comprobar que la compresión no altera los datos originales, se compararon dos representaciones del mismo estado de la partida:

1. El archivo de texto generado por `guardar`.
2. El contenido recuperado desde el archivo binario generado por `guardar_comprimido`.

Ambos deben corresponder exactamente a la misma representación textual.

Una segunda comprobación consiste en inicializar la partida desde el archivo comprimido y consultar el estado de los territorios.

De esta forma se verifica tanto el contenido recuperado como su interpretación dentro del sistema RISK.

6.12.8. Registro de resultados obtenidos

Los nueve casos del plan de pruebas fueron ejecutados utilizando la implementación correspondiente a la Entrega 2.

La Tabla 8 relaciona los valores de entrada, los resultados esperados y los resultados obtenidos, siguiendo el formato trabajado en el Taller 0 [2].

Las evidencias gráficas de estas ejecuciones se presentan junto a los resultados en las tablas del apartado 6.12.9.

Tabla 8: Plan de pruebas y resultados del comando guardar_comprimido.

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
GC01: partida sin inicializar	`guardar_comprimido` antes de ejecutar `inicializar`.	Rechazar el guardado e impedir la creación del archivo.	Se informó que la partida no estaba inicializada. No se generó el archivo.
GC02: guardado válido	Inicializar desde `partida.txt` y guardar en un nuevo archivo binario.	Informar el éxito del guardado y crear un archivo no vacío.	Se informó el éxito y se creó un archivo de 519 bytes.
GC03: partida modificada	Inicializar, ejecutar `obtener_unidades Isa` y asignar 4 unidades al territorio 1.2.	Guardar las unidades actualizadas y conservarlas después de recuperar el archivo.	El territorio pasó de 3 a 7 unidades. La recuperación mantuvo las 7 unidades y ambos textos coincidieron.
GC04: consistencia del texto	Guardar el mismo estado en un archivo de texto y en un binario. Recuperar el contenido.	El texto original y el recuperado deben ser idénticos.	La comparación `cmp` confirmó que ambos archivos de texto eran idénticos, con 511 bytes cada uno.
GC05: guardado repetido	Comprimir dos veces una misma partida sin modificar su estado intermedio.	Los archivos deben ser equivalentes y, con el desempate determinista, idénticos byte a byte.	La comparación con `cmp` confirmó que los dos binarios eran idénticos byte a byte.
GC06: error de escritura	Guardar el archivo binario en un directorio inexistente.	Informar un error de codificación o escritura y no crear el archivo.	El programa mostró el error esperado. Se comprobó que el archivo no se creó.

Tabla 8: Continuación del plan de pruebas de guardar_comprimido.

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
GC07: recuperación correcta	Inicializar una nueva ejecución desde `gc04_guardado.bin` y consultar la partida.	Aceptar el archivo comprimido válido y recuperar los datos de la partida.	La inicialización fue correcta. Se consultó el estado y se guardó nuevamente en texto.
GC08: encabezado inválido	Modificar la cabecera de una copia del archivo binario e intentar inicializar.	Rechazar el archivo por no cumplir el formato esperado.	El programa informó *Archivo sin formato* y rechazó la carga.
GC09: archivo truncado	Conservar solo 12 bytes de una copia del archivo comprimido e intentar inicializar.	Detectar la información incompleta y rechazar la inicialización.	El programa informó *Archivo sin formato* y rechazó la carga.

Los nueve casos obtuvieron los resultados esperados para las entradas utilizadas.

En particular, GC03 y GC04 permitieron comprobar que la recuperación conservó exactamente los datos territoriales evaluados.

El archivo comprimido de GC04 ocupó 519 bytes, mientras que su representación textual ocupó 511 bytes.

Por tanto, las pruebas demostraron recuperación correcta, pero no una reducción de tamaño para esa entrada.

Debe distinguirse el archivo utilizado para iniciar una partida de la representación textual que posteriormente produce el programa. Esta última conserva jugadores, propietarios y unidades, pero puede normalizar los saltos de línea y el formato de los registros.

En GC04, el archivo de texto guardado y el archivo recuperado desde Huffman contenían 511 bytes cada uno, y se comprobaron idénticos con `cmp`. El archivo comprimido ocupó 519 bytes. Por tanto, la comparación de integridad se refiere a los dos archivos de 511 bytes generados en el ensayo, no a una comparación automática contra el archivo usado inicialmente para cargar la partida.

Las pruebas GC08 y GC09 también confirmaron que el programa rechazó los dos archivos inválidos preparados para la evaluación.

6.12.9. Evidencias de ejecución

Se registran los casos ejecutados y las capturas de terminal que respaldan sus resultados. El criterio de presentación sigue el Taller 0: descripción de caso y evidencia de ejecución en columnas contiguas.

Las capturas fueron proporcionadas por el equipo y corresponden a pruebas del programa; no se han recreado salidas de consola para el documento. GC05 y GC06 comparten una captura de verificación porque contiene los resultados de ambos casos.

Tabla 9 Registro de evidencias de GC01 y GC02.

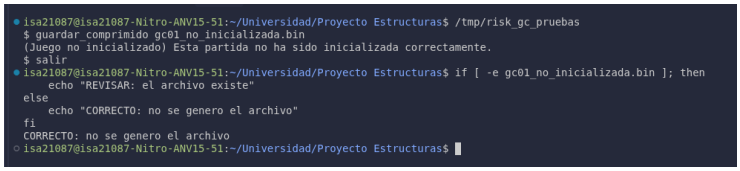
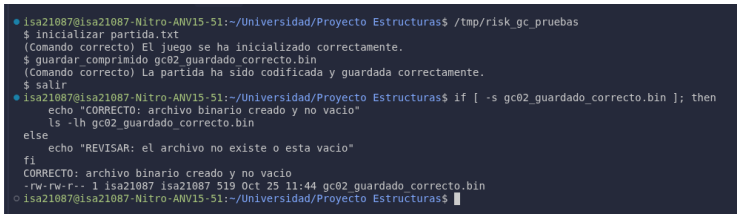
Descripción de caso	Evidencia de ejecución
GC01: partida no inicializada.	
GC02: guardado comprimido correcto.	

Tabla 10 Registro de evidencias de GC03: estado modificado y recuperado.

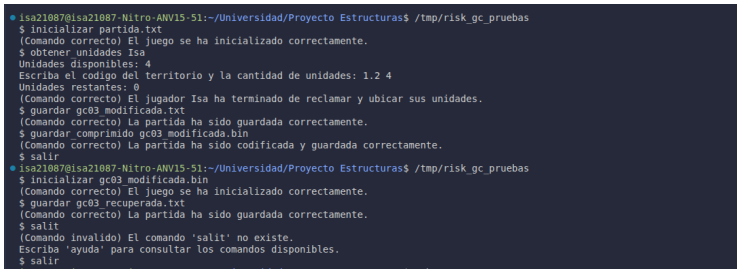
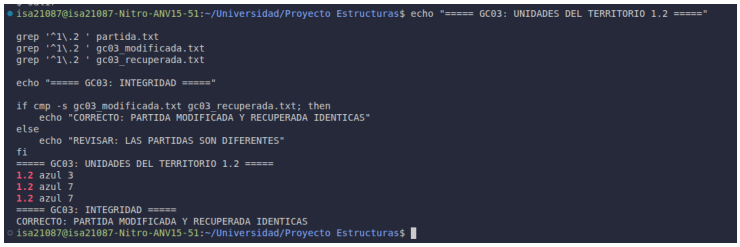
Descripción de caso	Evidencia de ejecución
GC03: agregar cuatro unidades y recuperar la partida.	 

Tabla 11 Registro de evidencias de GC04 y GC05.

Descripción de caso	Evidencia de ejecución

GC04: comparar archivos original y recuperado.	`isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$ echo "===== COMPARACION GC04 Y GC07 ====="`
	`if cmp -s gc04_original.txt gc07_recuperado.txt; then`
	`  echo "CORRECTO: TEXTO ORIGINAL Y RECUPERADO IDENTICOS"`
	`else`
	`  echo "REVISAR: LOS ARCHIVOS SON DIFERENTES"`
	`  diff -u gc04_original.txt gc07_recuperado.txt`
	`fi`
	`echo "===== TAMANOS ====="`
	`ls -lh gc04_original.txt gc04_guardado.bin gc07_recuperado.txt`
	`===== COMPARACION GC04 Y GC07 =====`
	`CORRECTO: TEXTO ORIGINAL Y RECUPERADO IDENTICOS`
	`===== TAMANOS =====`
	`-rw-rw-r-- 1 isa21087 isa21087 519 Oct 25 11:48 gc04_guardado.bin`
	`-rw-rw-r-- 1 isa21087 isa21087 511 Oct 25 11:48 gc04_original.txt`
	`-rw-rw-r-- 1 isa21087 isa21087 511 Oct 25 11:54 gc07_recuperado.txt`
	`isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$`

GC05: dos guardados idénticos, con verificación byte a byte.	`isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$ echo "===== GC04 ====="`
	`ls -lh gc04_original.txt gc04_guardado.bin`
	`echo "===== GC05 ====="`
	`if cmp -s gc05_a.bin gc05_b.bin; then`
	`  echo "GC05: ARCHIVOS IDENTICOS BYTE A BYTE"`
	`else`
	`  echo "GC05: ARCHIVOS DIFERENTES"`
	`fi`
	`echo "===== GC06 ====="`
	`if [ -e gc06_directorio_inexistente/fallo.bin ]; then`
	`  echo "GC06: REVISAR"`
	`else`
	`  echo "GC06: NO SE CREO EL ARCHIVO"`
	`echo "===== GC05 ====="`
	`if cmp -s gc05_a.bin gc05_b.bin; then`
	`  echo "CORRECTO: ARCHIVOS IDENTICOS BYTE A BYTE"`
	`else`
	`  echo "REVISAR: ARCHIVOS DIFERENTES"`
	`fi`
	`echo "===== GC06 ====="`
	`if [ -e gc06_directorio_inexistente/fallo.bin ]; then`
	`  echo "REVISAR: EL ARCHIVO EXISTE"`
	`else`
	`  echo "CORRECTO: NO SE CREO EL ARCHIVO"`
	`fi`
	`===== GC05 =====`
	`CORRECTO: ARCHIVOS IDENTICOS BYTE A BYTE`
	`===== GC06 =====`
	`CORRECTO: NO SE CREO EL ARCHIVO`
	`isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$`

Tabla 12 Registro de evidencias de GC06: manejo de error de escritura.

Descripción de caso	Evidencia de ejecución

GC06: directorio inexistente y ausencia del archivo de salida.	`$ guardar gc04_original.txt`
	`(Comando correcto) La partida ha sido guardada correctamente.`
	`$ guardar_comprimido gc04_guardado.bin`
	`(Comando correcto) La partida ha sido codificada y guardada correctamente.`
	`$ guardar_comprimido gc05_a.bin`
	`(Comando correcto) La partida ha sido codificada y guardada correctamente.`
	`$ guardar_comprimido gc05_b.bin`
	`(Comando correcto) La partida ha sido codificada y guardada correctamente.`
	`$ guardar_comprimido gc06_directorio_inexistente/fallo.bin`
	`(Error al codificar y/o guardar) La partida no ha sido codificada ni guardada correctamente.`
	`echo "===== GC05 ====="`
	`if cmp -s gc05_a.bin gc05_b.bin; then`
	`  echo "CORRECTO: ARCHIVOS IDENTICOS BYTE A BYTE"`
	`else`
	`  echo "REVISAR: ARCHIVOS DIFERENTES"`
	`fi`
	`echo "===== GC06 ====="`
	`if [ -e gc06_directorio_inexistente/fallo.bin ]; then`
	`  echo "REVISAR: EL ARCHIVO EXISTE"`
	`else`
	`  echo "CORRECTO: NO SE CREO EL ARCHIVO"`
	`fi`
	`===== GC05 =====`
	`CORRECTO: ARCHIVOS IDENTICOS BYTE A BYTE`
	`===== GC06 =====`
	`CORRECTO: NO SE CREO EL ARCHIVO`
	`isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$`

Tabla 13 Registro de evidencias de GC07 y GC08.

Descripción de caso	Evidencia de ejecución

GC07: recuperación y consulta desde archivo binario.	/tmp/risk_gc_pruebas
	$ inicializar gc04_guardado.bin
	(Comando correcto) El juego se ha inicializado correctamente.
	$ guardar gc07_recuperado.txt
	(Comando correcto) La partida ha sido guardada correctamente.
	$ estado_juego
	========== ESTADO DEL JUEGO ==========
	Cantidad de jugadores: 3
	Jugador: Isa | Color: azul
	Jugador: Saul | Color: verde
	Jugador: Alejo | Color: rojo
	Turno actual: Isa

GC08: rechazo de encabezado inválido.	isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$ /tmp/risk_gc_pruebas
	$ inicializar gc08_cabecera_invalida.bin
	(Archivo sin formato) gc08_cabecera_invalida.bin no contiene información en el formato esperado.
	$ salir
	isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$

Tabla 14 Registro de evidencias de GC09.

Descripción de caso	Evidencia de ejecución

GC09: rechazo de archivo binario truncado.	isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$ /tmp/risk_gc_pruebas
	$ inicializar gc09_truncado.bin
	(Archivo sin formato) gc09_truncado.bin no contiene información en el formato esperado.
	$ salir
	isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Proyecto Estructuras$

7. Conclusiones

El desarrollo incremental del proyecto RISK permite integrar las estructuras de datos estudiadas en los diferentes cortes de la asignatura dentro de un único sistema.

En el primer componente se utilizaron estructuras lineales para representar jugadores, cartas, territorios, continentes y los elementos necesarios para administrar una partida.

El diseño de estos elementos se organizó mediante Tipos Abstractos de Datos, distinguiendo el estado, las condiciones de las variables y las operaciones de cada estructura.

La segunda entrega amplía este diseño mediante la incorporación de estructuras jerárquicas utilizadas en la codificación de Huffman.

El árbol binario permite representar los símbolos del mensaje y sus frecuencias, mientras que el montículo proporciona el mecanismo de selección por prioridad necesario para construirlo.

La integración con el TAD Partida permite reutilizar la representación textual del juego para el guardado normal y el almacenamiento comprimido.

Asimismo, la recuperación de archivos binarios se conecta con el procedimiento de inicialización existente, evitando mantener procesos independientes para validar los mismos datos.

Desde el punto de vista del diseño, esta organización permite conservar las responsabilidades del primer componente y agregar las operaciones de almacenamiento sin sustituir las estructuras previamente implementadas.

La documentación de pruebas también mantiene un enfoque incremental.

Se conservaron los doce casos registrados para el comando `atacar` en la Entrega 1, distinguiendo sus resultados históricos de las verificaciones correspondientes al estado actual del proyecto.

Para la Entrega 2 se definió un plan de pruebas específico para el comando `guardar_comprimido`, incluyendo casos normales, de error y de consistencia.

Se ejecutaron nueve casos de prueba del almacenamiento comprimido, incluyendo condiciones normales, errores y recuperación de archivos.

Los resultados observados coincidieron con los esperados en los nueve casos y quedaron asociados con sus respectivas evidencias de ejecución.

Finalmente, la documentación reúne los componentes desarrollados hasta la segunda entrega y constituye la base para incorporar

posteriormente las estructuras de grafos y los algoritmos correspondientes al tercer componente.

8. Alcance de la documentación y verificaciones finales

El presente documento describe el diseño de los Componentes 1 y 2 del proyecto RISK.

Los resultados de pruebas correspondientes a la Entrega 1 se conservan como evidencia documental histórica.

Los nueve casos de prueba del almacenamiento comprimido fueron ejecutados y documentados con sus resultados y evidencias.

La revisión documental contrastó la interfaz de los TAD con las declaraciones del código de la Entrega 2. Se identificó como auxiliar privada la función `obtenerTexto Partida()` y se conservó la distinción entre el módulo de funciones Huffman y los TAD Árbol Binario y Cola de Prioridad.

Se compiló una copia del código C++ proporcionado por el equipo con `g++` en modo C++17. La compilación produjo un ejecutable, aunque se registraron advertencias relacionadas principalmente con comparaciones entre tipos enteros con y sin signo. Esta comprobación de compilación no equivale a ejecutar todos los casos de uso ni sustituye las pruebas de la versión exacta que se entregue.

Como verificación técnica adicional, separada de GC01–GC09, se utilizó la copia de `partida.txt` incluida en el archivo histórico de la Entrega 1. Se cargó la partida, se generaron archivos de texto (511 bytes) y Huffman (519 bytes), se recuperó el comprimido en una nueva ejecución y se compararon los textos con `cmp`: fueron idénticos byte a byte. Una copia binaria truncada a 12 bytes se rechazó con el estado Archivo sin formato. Esta comprobación corresponde a la versión del código revisada localmente, no a un nuevo registro histórico del equipo.

La documentación presenta por separado las pruebas históricas P01–P12 y las evidencias GC01–GC09 aportadas por el equipo. No se atribuyen ejecuciones no registradas.

Limitación funcional detectada. El proceso de conquista permite que un territorio cambie de propietario con cero unidades, mientras que `inicializar()` rechaza territorios con menos de una unidad. Esto puede impedir recuperar un estado válidamente guardado según la regla de conquista que admite la implementación. La diferencia queda registrada como riesgo técnico y no se presenta como corregida en el código de esta entrega.

La presente documentación no incluye como desarrolladas las estrategias de conquista basadas en grafos, ya que corresponden al tercer componente del proyecto.

9. Uso de herramientas de inteligencia artificial

Durante el desarrollo y la documentación de RISK se utilizó ChatGPT como apoyo para consultar conceptos de C++, contrastar especificaciones con el material de clase y revisar la integración del almacenamiento y de Huffman. La asistencia incluyó propuestas de código revisadas y adaptadas por el equipo, así como apoyo en la organización y redacción de secciones, tablas y explicaciones en LaTeX.

Las intervenciones en el código se identifican mediante comentarios junto a los bloques correspondientes. En el archivo fuente LaTeX también se señalan los apartados reorganizados con asistencia y se describen las consultas realizadas. Las consultas reconstruidas posteriormente no se presentan como prompts originales literales.

La verificación final de las atribuciones y del alcance de la asistencia corresponde al equipo, conforme al apartado 3.5 del enunciado oficial.

A. Construcción del árbol de Huffman: ejemplo actualizado

Los diagramas de este anexo proceden de la versión actualizada de la documentación técnica elaborada por el equipo para el segundo componente. Se incluyen las 26 etapas de construcción del árbol que aparecen en esa versión, y no la secuencia anterior de 22 imágenes.

El ejemplo emplea 27 símbolos distintos. La suma de las frecuencias registradas es 511, valor que coincide con la frecuencia acumulada de la raíz del árbol final. La coincidencia de esa longitud con el texto observado en la prueba GC04 no prueba, por sí sola, que ambos contenidos sean idénticos; los diagramas ilustran el algoritmo y no reemplazan la comparación experimental de integridad.

La Tabla 15 reproduce las frecuencias registradas en la versión actualizada de la documentación del equipo. Se distingue el espacio del salto de línea para evitar confusiones.

Tabla 15 Histograma del ejemplo actualizado de construcción de Huffman.

Símbolo	Frecuencia	Símbolo	Frecuencia
Espacio	87	`j`	16
`\n`	45	`u`	16
`.`	42	`d`	15
`3`	35	`v`	15
`2`	32	`z`	15
`e`	31	`4`	12
`o`	31	`6`	8
`r`	30	`7`	3
`1`	19	`8`	2
`a`	17	`9`	2
`l`	17	`0`	1
`5`	16	`A`	1
`I`	1	`S`	1
`s`	1		
		Total	**511**

El proceso sigue la selección de los dos nodos con menor frecuencia disponible, su unión bajo un nuevo nodo padre y la reinserción de este último en el montículo. En los casos de empate, la configuración concreta de ramas depende de la regla de desempate aplicada por la implementación.

A.1. Interpretación de los diagramas

Los diagramas permiten observar la organización jerárquica de las frecuencias durante la construcción de Huffman.

Inicialmente, cada símbolo tiene asociado un nodo hoja.

En cada iteración se seleccionan las dos raíces de menor frecuencia disponibles.

Ambas se conectan como descendientes de un nodo padre cuya frecuencia es la suma de las frecuencias de sus hijos.

El nuevo subárbol vuelve a incorporarse a la cola de prioridad.

El procedimiento continúa hasta obtener una única raíz.

A partir del árbol completo, cada símbolo puede asociarse con una secuencia de bits determinada por su camino desde la raíz hasta la hoja.

En la implementación del proyecto, la secuencia exacta de uniones también depende del criterio utilizado para resolver empates entre frecuencias iguales.

Por ello, la validación del algoritmo no debe basarse únicamente en la apariencia del árbol.

Se comprueba principalmente que el archivo tenga el formato exigido y que la descompresión permita recuperar exactamente la información original.

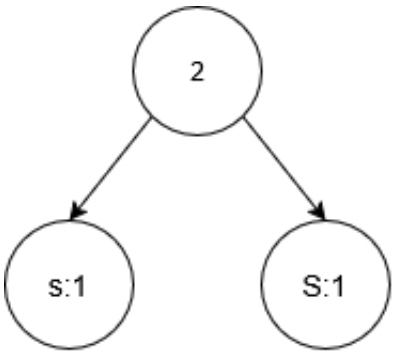


Diagrama 1

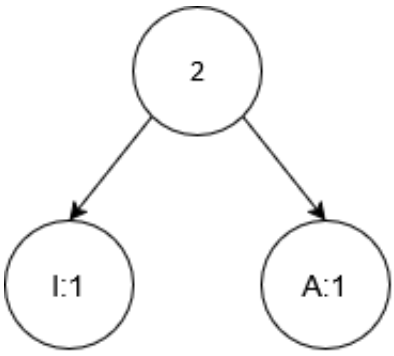


Diagrama 2

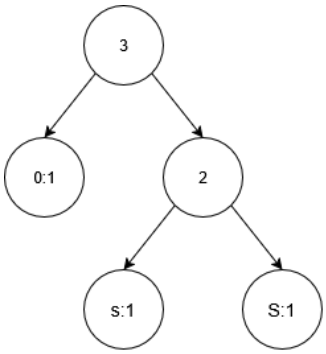


Diagrama 3

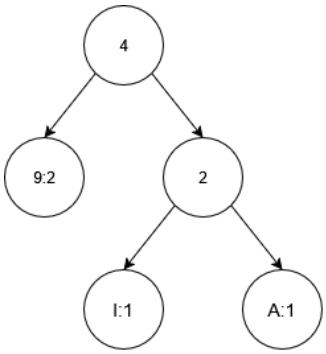


Diagrama 4

Figura 8 Etapas 1 a 4 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

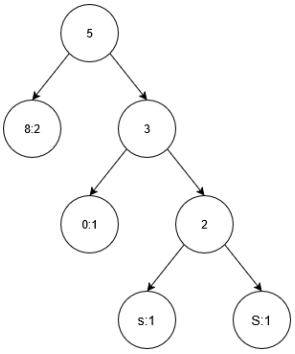


Diagrama 5

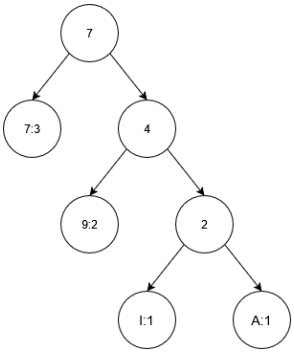


Diagrama 6

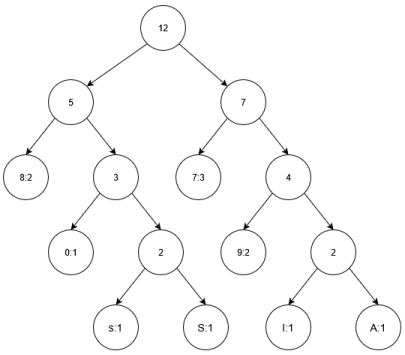


Diagrama 7

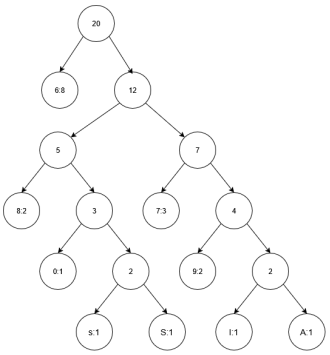


Diagrama 8

Figura 9 Etapas 5 a 8 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

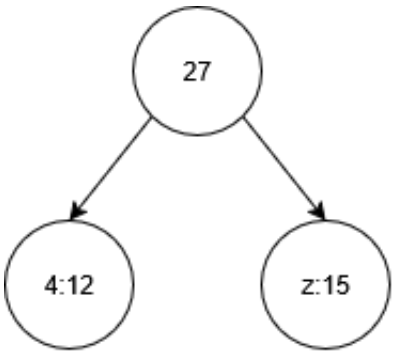


Diagrama 9

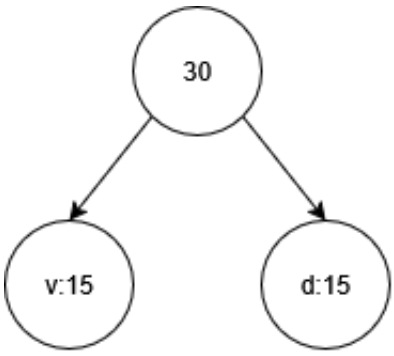


Diagrama 10

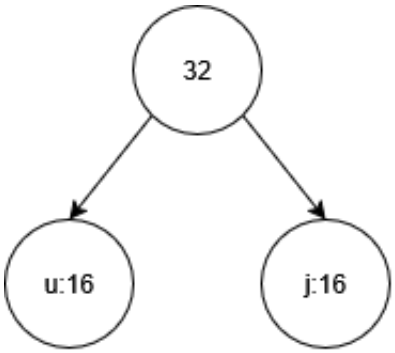


Diagrama 11

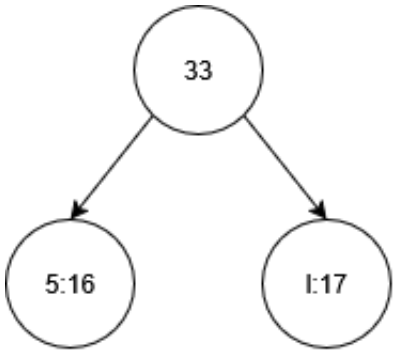


Diagrama 12

Figura 10 Etapas 9 a 12 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

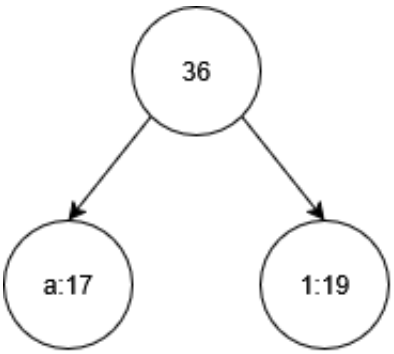


Diagrama 13

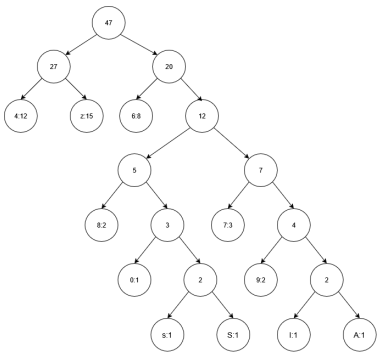


Diagrama 14

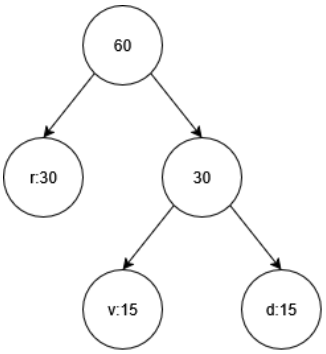


Diagrama 15

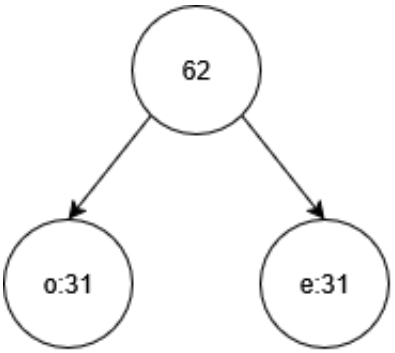


Diagrama 16

Figura 11 Etapas 13 a 16 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

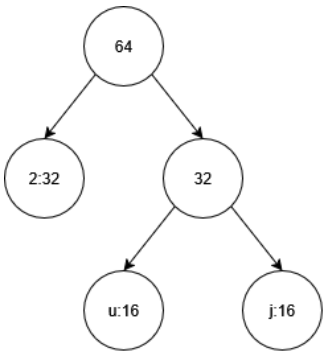


Diagrama 17

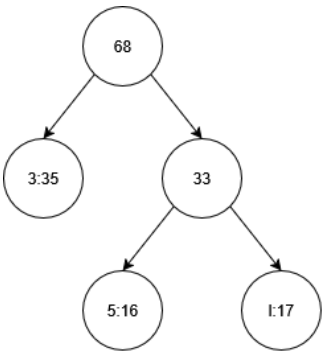


Diagrama 18

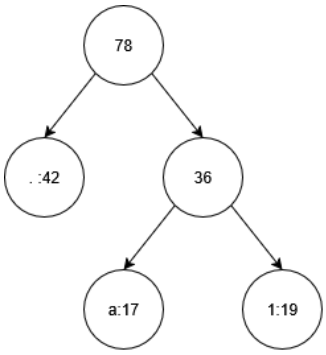


Diagrama 19

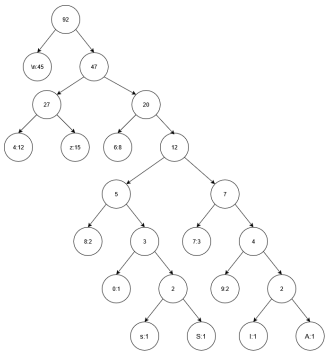


Diagrama 20

Figura 12 Etapas 17 a 20 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

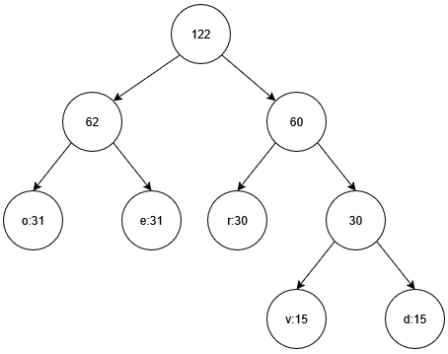


Diagrama 21

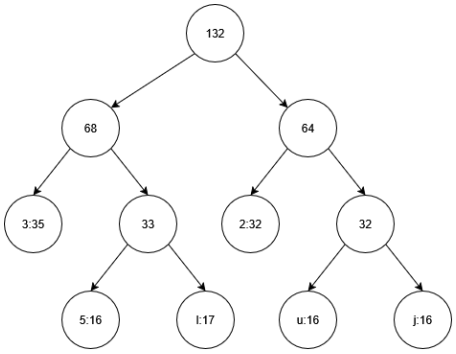


Diagrama 22

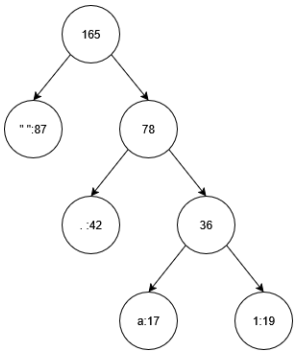


Diagrama 23

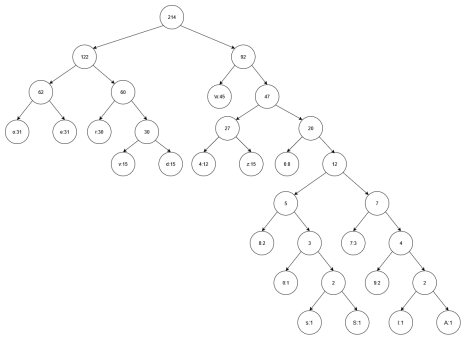


Diagrama 24

Figura 13 Etapas 21 a 24 de la construcción del árbol de Huffman en la documentación actualizada del equipo.

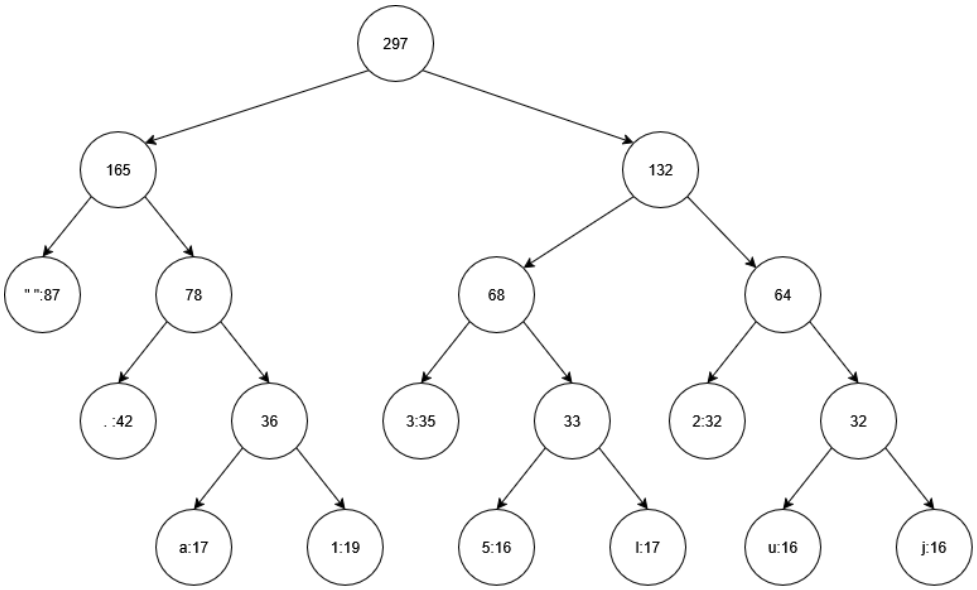


Figura 14 Diagrama 25. Uniones avanzadas entre subárboles, previas al árbol completo.

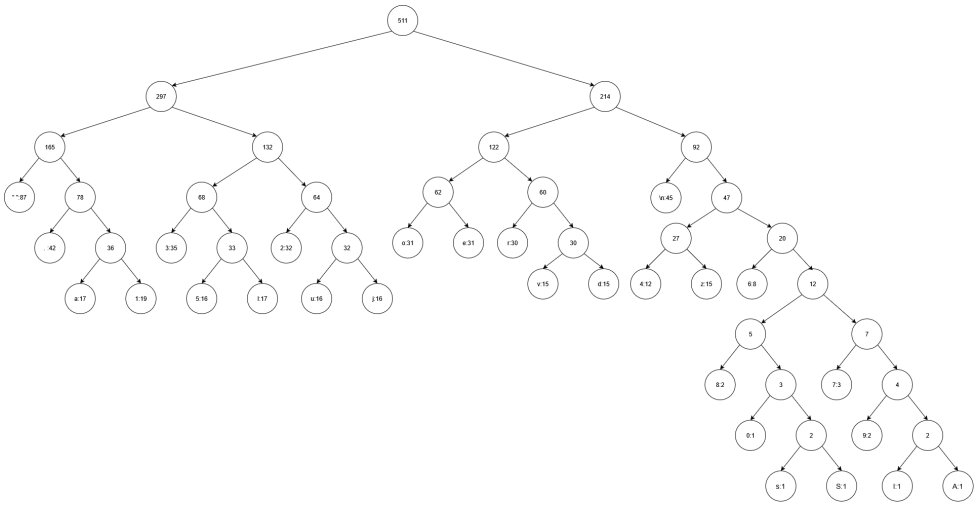


Figura 15 Diagrama 26. Árbol de Huffman final del ejemplo revisado: la frecuencia de la raíz es 511, igual a la suma de las frecuencias del histograma.

Referencias

[1] Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas. *Proyecto del curso 2026-30*. Estructuras de Datos, 2026.

[2] Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas. *Taller 0: Compilación, pruebas y depuración*. Estructuras de Datos, 2026-30.

[3] Castro Martínez, A. *Tipo Abstracto de Dato (TAD)*. Material de clase de Estructuras de Datos, Pontificia Universidad Javeriana, 2023.

[4] Castro Martínez, A. *Secuencias; Vectores y Listas*. Material de clase de Estructuras de Datos, Pontificia Universidad Javeriana, 2023.

[5] Cruz Romero, S. L., Hermosa Losada, I. y Macías Barrios, A. *Taller 2: Sistema de batallas Pokémon*. Pontificia Universidad Javeriana, 2026.

[6] Pontificia Universidad Javeriana. *08: Árboles Binarios*. Diapositivas de la asignatura Estructuras de Datos, material de clase.

[7] Pontificia Universidad Javeriana. *11: Montículos*. Diapositivas de la asignatura Estructuras de Datos, material de clase.

[8] Pontificia Universidad Javeriana. *13: Expresión y Codificación*. Diapositivas de la asignatura Estructuras de Datos, material de clase.

[9] Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas. *Taller 3: Árboles de búsqueda*. Estructuras de Datos, 2026-30.

[10] cppreference.com. *std::ios_base::openmode: modos de apertura de flujos C++*. Disponible en: https://en.cppreference.com/w/cpp/io/ios_base/openmode. Consultado como referencia técnica de implementación, 2026.

[11] cppreference.com. *std::basic_ostream::write: escritura de bloques de caracteres*. Disponible en: https://en.cppreference.com/w/cpp/io/basic_ostream/write. Consultado como referencia técnica de implementación, 2026.

[12] cppreference.com. *std::basic_istream::read: lectura de bloques de caracteres*. Disponible en: https://en.cppreference.com/w/cpp/io/basic_istream/read. Consultado como referencia técnica de implementación, 2026.

[13] cppreference.com. *std::basic_istringstream: entrada de datos desde cadenas*. Disponible en: https://en.cppreference.com/w/cpp/io/basic_istringstream. Consultado como referencia técnica de implementación, 2026.

[14] cppreference.com. *std::basic_ostringstream: salida de datos hacia cadenas*. Disponible en: https://en.cppreference.com/w/cpp/io/basic_ostringstream. Consultado como referencia técnica de implementación, 2026.

[15] cppreference.com. *std::push_heap: inserción de elementos en un montículo.* Disponible en: https://en.cppreference.com/w/cpp/algorithm/push_heap. Consultado como referencia técnica de implementación, 2026.

[16] cppreference.com. *std::pop_heap: extracción de elementos de un montículo.* Disponible en: https://en.cppreference.com/w/cpp/algorithm/pop_heap. Consultado como referencia técnica de implementación, 2026.